

2. 四个设计阶段

- a. 数据设计或类设计data/class design：将类模型转化为**设计类的实现以及软件实现所要求的数据结构**。——CRC图中定义的对象和关系、类属性和其他表示法刻画的详细数据内容为数据设计活动提供了基础。
- b. 体系结构设计architectural design：定义了软件的主要构造元素之间的关系、可用于达到系统所定义需求的**体系结构风格和设计模式**以及影响体系结构实现方式的**约束**。——体系结构设计表示可以从**需求模型**导出。
- c. 接口设计interface design：描述了**软件和协作系统之间、软件和使用人员之间是如何通信的**。——使用场景和行为模型为接口设计提供了所需的大量信息。
- d. 构件级设计component-Level Design：将软件**体系结构的构造元素**变换为**对软件构件的过程性描述**。——从**基于类的模型、流模型和行为模型**获得的信息作为构件设计的基础。

3. 五个设计模型

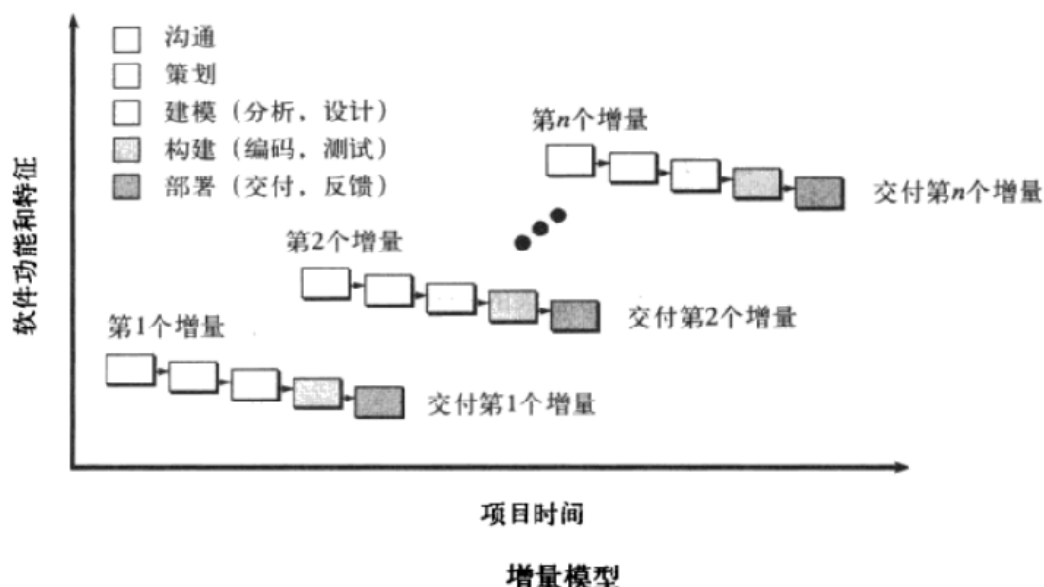
- a. 数据设计元素：将用户视角的数据模型逐步精化为可实现的表示形式。
- b. 体系结构设计元素：提供软件整体结构视图
- c. 接口设计元素：定义信息流内/外部通信机制
- d. 构件级设计元素：细化构件内部细节
- e. 部署级设计元素：分配功能到**物理计算环境**

2. 单元测试、集成测试、确认测试和系统测试，每一层的测试的目标是什么？



- 1. 单元测试Unit Testing：验证软件**单个构件的内部正确性**，确保它编码实现的和算法设计一致且构建起到了单元的作用。
- 2. 集成测试 Integration testing：利用已通过单元测试的构件**建立程序结构**，目的是**处理并验证与程序构造相关的问题**，确保**集成后的交互正确性**
- 3. 确认测试Validation testing：按照**需求说明书**，确保所有的功能、行为和性能需求在软件产品里面已经实现。
- 4. 系统测试System testing：确保软件在**完整计算机系统中**正确集成、稳定运行、完成功能。（通过恢复、安全、压力、性能、部署五类测试分别验证其容错性、安全性、鲁棒性、效率及环境适应性。）

3. There are five activities in generic process framework.Please apply them to describe the incremental model 增量模型.



用过程框架描述：

1. **沟通 (Communication):** 在每个增量的开始阶段，开发团队都会与客户进行沟通。对于第一个增量，沟通的重点是理解项目的核心需求，定义出最基本的功能，即“核心产品”。对于后续的增量，沟通则侧重于获取客户对上一个已交付增量的反馈和评价，并确定下一个增量需要新增或完善的功能。
2. **策划 (Planning):** 团队进行策划活动，对当前增量的工作进行任务分解、资源分配和制定详细的开发计划，确保每个增量的交付都是可控和可预测的。
3. **建模 (Modeling):** 团队对当前增量需要实现的需求进行分析和设计，建立分析和设计模型。
4. **构建 (Construction):** 根据模型进行编码测试，构建出一个可运行、可交付、具备了用户服务能力的软件增量。
5. **部署 (Deployment):** 可运行的软件增量会被交付（部署）给客户。客户可以使用这个增量，并提供反馈。这个反馈将成为下一个增量“沟通”活动的输入。

什么条件下使用：

1. **初始需求明确，但需要快速交付一个可用版本时：** 当软件的初始需求有比较清晰的定义，同时市场或用户迫切需要快速得到一个虽然功能有限但可用的核心产品时，增量模型非常适用。后续的功能可以在之后的版本中逐步添加。
2. **开发人力资源不足，无法一次性投入全部人员时：** 用少量人员先开发并交付核心产品，如果产品反馈良好，可以为后续的增量投入更多的人力资源。
3. **项目中存在不确定的技术风险（某些技术或硬件组件存在交付不确定性）时：** 开发团队可以在早期的增量中先不使用这些不确定的技术，从而保证核心功能可以按时交付，避免整个项目被延期。

a. **Incremental process** models tend to be among the most widely used in the industry. Describe the conditions under which such models should be used and what an "increment" means in terms of project work and delivery. 描述应该在什么条件下使用这种模型，以及“增量”在项目工作和交付方面的含义。



“增量”在项目工作和交付方面的含义：

- 每个增量是一个独立的线性开发序列，包含完整的沟通策划建模构建部署的过程。每个增量可采用不同开发模型，增量间顺序推进。
- 每个增量均交付一个可运行的软件子集，具备实际用户服务能力。早期增量是最终产品的“片段版本”，但功能完整可用。每次交付都让用户获得部分功能，并为后续改进提供反馈平台。

优势：

- 每一次迭代开发得到的增量都是一个可运行的软件版本，这提高了软件开发过程的可见性，有利于快速交付部分功能抢占市场，收集用户反馈意见，改进软件的下一个版本
- 可以规避风险，用小批量的增量进行尝试，成功以后再在以后的迭代中更大范围地推广经验

4. There are five activities in generic process framework. Please apply them to describe the **waterfall model**.

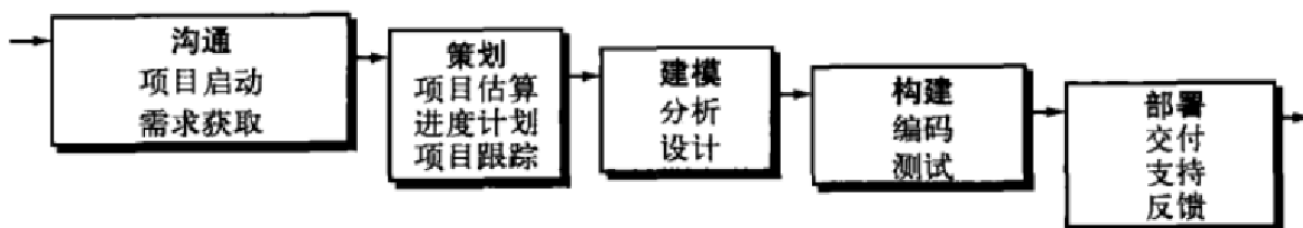


图2-3 瀑布模型



For the waterfall model, describe situations where this model can and cannot be used and why. 对于瀑布模型，请描述该模型适用和不适用的情况，并说明原因。

- 适用于：需求明确、固定且项目工作可以采用线性方式完成的场景。

原因：瀑布模型是一个系统、顺序的过程，它要求在项目开始时就确定所有需求。当需求明确且稳定时，这种线性方法能高效地推进，避免因变更导致的混乱。结构化特性有助于确保每个阶段的完整性，从而减少后期错误，并提供清晰的项目里程碑。

- 不适用于需求不确定、易变、后期可能需要进行大量修改、项目涉及快速变化的场景。

原因：由于瀑布模型按照阶段进行分类，只有完成当前阶段才能开始下一个阶段。在需求不明确的项目中，各个阶段之间反馈很少，客户必须等到项目结束时才能获得可执行程序，过程中无法修改，这会导致修改后期阶段非常复杂，并且可能导致重大缺陷晚被发现，造成严重损失。

描述：

沟通：包括**项目启动**和**需求获取**。客户清晰、完整地描述所有需求。此阶段的产出（如《需求规格说明书》）将被“冻结”，作为后续所有阶段的唯一依据。

策划：包括**项目估算**、**进度计划**和**项目跟踪**。

建模：系统架构设计、数据库设计、模块接口设计等。在不编写任何代码之前，创建出一套完整的系统蓝图。

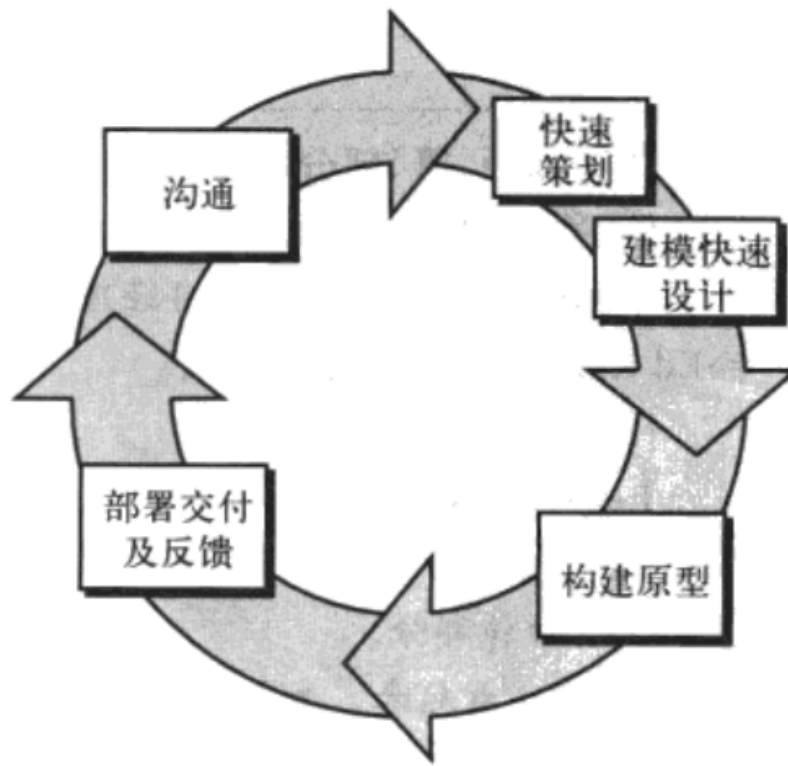
构建：根据建模阶段产出的详细设计文档，进行代码编写。测试人员对代码进行单元测试和集成测试，以验证软件是否符合设计要求。

部署：当软件构建和测试全部完成后，它作为一个完整的最终产品被交付给客户使用。

Level 2

1. Describe the phases of the **prototyping model** for software development. 描述软件开发的原型模型的阶段

- 原型开发即，简化软件的实现，只实现软件的部分功能，借用软件工具快速生成一个软件样品，这个样品就是软件原型，然后基于原型与用户进行沟通，确定修改意见，再对原型进行修改和增加功能。通过多次演化开发，最终生成满足用户需求的完整软件。
- 在每次进化过程中，采用瀑布模型进行开发。
- 适合需求不清楚的系统

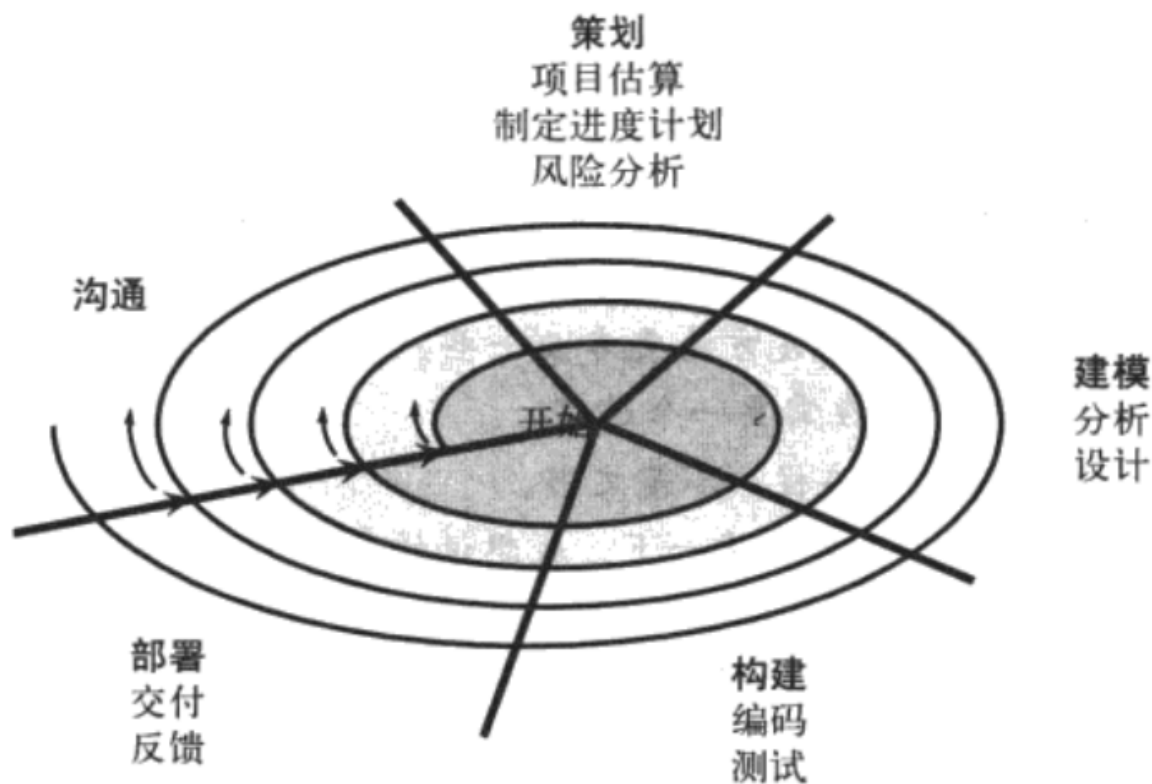


1. 沟通：软件开发人员与利益相关者会晤，共同完成：定义软件整体目标、明确已知需求、勾画需进一步定义的模糊需求
2. 快速策划：迅速且轻量级的规划即将开始的原型开发迭代，而不是制定一个长期的、详尽的项目计划。
3. 建模快速设计：聚焦用户可见部分，忽略后台逻辑细节，快速设计，例如人机交互界面的布局或数据显示的格式。
4. 构建原型：利用现有工具或程序片段，快速实现可执行原型。
5. 部署交付与反馈：将原型交付给利益相关者使用，利益相关者基于实际体验提出修改建议。

需求迭代与演进：根据反馈细化需求，并决定后续动作——若原型仅用于需求验证则废弃原型，部分原型可能演化成实际系统

ps：若需求仍不明确，重复阶段1-4直至共识达成。

2. There are five activities in generic process framework. Please apply them to describe the **spiral process**.



1. 沟通：在每个迭代周期的开始，软件团队与客户及其他利益相关者进行沟通。在项目初期，沟通的目的是为了建立一个“概念开发项目”；在后续的迭代中，沟通则是为了获取对上一个版本的反馈，并明确当前迭代要开发的新特性或完善的功能。
 2. 策划：不仅包括常规的**项目估算**和**制定进度计划**，最核心的是进行**风险分析**。团队会识别并分析当前迭代中可能遇到的技术风险和管理风险，并制定应对策略。每次螺旋循环都会重新进行策划，根据上一次交付后的用户反馈来调整项目计划、预算和进度。
 3. 建模：根据当前迭代的目标，团队进行系统分析和软件设计。在早期的迭代中，建模的产出可能只是一个理论模型或一个简单的原型设计；在后期的迭代中，则会是更完整、更详细的系统架构和模块设计。
 4. 构建：开发团队根据建模阶段的设计进行**编码**和**测试**。每一圈螺旋迭代都会产生出一个可运行的软件版本，这个版本比前一个更完善。
 5. 部署：当前迭代构建出的软件版本被**交付**给客户进行评估，并收集其**反馈**。直接用于下一圈螺旋迭代开始时的“沟通”和“策划”活动，从而驱动软件的下一次演进。
- 独特之处在于，**风险分析**被置于每次迭代的**核心**

3. 统一过程模型UP



1. UP的起始阶段（inception phase）包括沟通和策划。通过与利益相关者协作定义软件的业务需求,提出系统大致的架构，并制定开发计划以保证项目开发具有迭代和增量的特性。
2. 细化阶段（Elaboration Phase）包括沟通和建模活动。细化阶段扩展了初始阶段定义的用例，并扩展了体系结构以包括软件的五种视图——用例模型、需求模型、设计模型、实现模型和部署模型。
3. UP的构建阶段（construction phase）与构建活动相同。构建阶段采用体系结构模型作为输入，开发或是获取软件构件，使得最终用户能够操作用例。
4. UP的转换阶段（transition phase）构建的后期以及部署的第一部分。软件被提交给最终用户进行 Beta 测试，用户反馈报告缺陷及必要的变更。在转换阶段结束时,软件增量成为可用的发布版本。
5. UP的生产阶段（production phase）与通用过程的部署活动一致。对持续使用的软件进行监控，提供运行环境（基础设施）的支持，提交并评估缺陷报告和变更请求。

五个UP阶段并不是顺序进行，而是阶段性地并发进行。

4. 通用过程框架是什么？（Describe the 5 phases of the Unified Process model for software engineering.）



• 五种框架活动——沟通、策划、建模、构建以及部署。

- 沟通Communication——与客户（及其他利益相关者）的沟通协调。了解利益相关者对于项目的目标，并收集有助于定义软件特性和功能的需求。
- 策划Planning——通过描述要执行的技术任务、可能出现的风险、所需的资源、要产生的工作产品以及工作进度表，来界定软件工程项目。
- 建模Modeling——分析和设计模型，绘制项目体系架构，利用模型理解软件需求。
- 构建Construction——编写代码，为发现代码中的错误所需的测试。
- 部署Deployment——软件（作为一个完整的实体或部分完成的增量版本）被交付给客户，客户会评估所交付的产品并基于评估提供反馈。

• 一系列普适性活动Umbrella activities跟风质技审度量佩服工

1. 软件项目跟踪与控制——使软件团队能够根据项目计划评估进展情况，并采取必要措施以维持进度。
2. 风险管理——评估可能影响项目结果或产品质量的风险。
3. 软件质量保证——界定并开展确保软件质量所需的活动。

4. 技术评审——对软件工作产品进行评估，力求在错误传播到下一活动之前发现并消除它们。
5. 度量——界定并收集过程、项目和产品度量数据，协助团队交付满足利益相关者需求的软件；可与所有其他框架活动和综合性活动结合使用。
6. 软件配置管理——在整个软件过程中管理变更的影响。
7. 可复用性管理——界定工作产品（包括软件组件）复用的标准，并建立实现可复用组件的机制。
8. 工作产品准备与制作——涵盖创建模型、文档、日志、表格和清单等工作产品所需的的活动。

5. What are the six steps for requirements engineering? 需求工程的六个步骤是什么？



起始。发现软件，陈述软件想法。

导出。软件调研，获取需求信息。

精化。分析获得需求模型。

规格说明。需求文档化，用自然语言和需求模型描述需求。

确认。检查是否采用正确需求工程过程和方法创建需求文档；检查需求文档是否满足质量标准。

协商。当需求存在冲突时，启动协商，达成一致理解。

需求管理。管理需求活动所产生的各种文档。使之具有可追溯性。

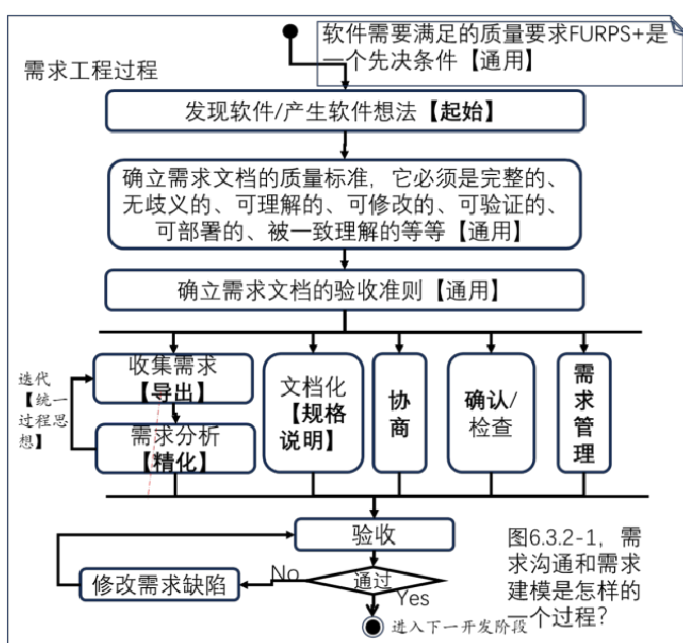


图6.3.2-1揭示了需求工程7项职责之间的关系。解释如下：

一，执行需求工程过程，将产生需求制品（称需求文档）。我们必须关注它们的质量，因为它是面向涉众的，且是后续开发活动的基准，一旦其质量出现问题，就可能偏离涉众的期望，从而软件失去价值。为此，需要在正式开始需求沟通和需求建模之间，先确立需求文档的质量标准和验收标准，以便行动具有明确的目标，使得创建行为——执行7项任务创建需求制品时，自然采用满足这些质量准则的正确流程和技术，而且也使得判断需求制品好坏——是否满足质量准则——提供判断标准，最终提供满足质量准则的需求制品。此处的质量标准具有普遍性，即任何软件项目皆如此。参见6.3.3理解需求文档的质量准则。

二，需求工程的主线活动是：发现软件（起始）、收集需求（导出）和需求分析（精化）；辅线活动是确认、协商、文档化（规格说明）和需求管理。其得到有效实施的先决条件是确立需求文档的质量准则和验收准则。三，在整个时间线上，需求收集和需求分析是同时进行的；只是，在早期，焦点活动是“需求收集”，之后，焦点活动向“需求分析”转移。这正是统一过程思想的体现。

四，无论发现软件，需求收集，还是需求分析，都要将活动的产物（称“需求制品”）用自然语言或模型语言将其陈述出来，这一活动称之为“规格说明”。

五，无论发现软件，需求收集，还是需求分析，都要对“需求制品”（文档化的需求）的质量进行确认。不仅如此，还要对产生需求制品的过程质量进行确认，即检查你是否按照需求工程或软件工程的过程和方法进行的开发活动。

六，一旦发生需求缺陷或冲突，则需要启动协商程序。

七，除了规格说明（文档化）、确认活动是贯穿性活动，需求管理也是贯穿性活动。

6. What work products result from the requirements engineering process? 需求工程过程会产生哪些工作成果？



1. 一份**需求与可行性说明**。
2. 一份关于系统或产品范围的**限定性说明**。
3. 参与需求获取的客户、用户及其他利益相关者的**名单**。
4. 对**系统技术环境**的描述。
5. 一份**需求列表**（最好按功能分类整理），以及适用于每项需求的**领域约束**。
6. 一套**使用场景**，用于展现系统或产品在不同操作条件下的使用情况。
7. 为更清晰定义需求而开发的任何**原型**。

Level3

1. 界面设计黄金原则



1. 让用户掌握控制权。
 - (1) 定义交互模式时，不应强迫用户执行不必要或不期望的操作。
 - (2) 提供灵活的交互方式。
 - (3) 允许用户交互可中断且可撤销。
 - (4) 随着用户技能水平提升，简化交互流程，并允许自定义交互方式。
 - (5) 对非专业用户隐藏技术内部细节。
 - (6) 设计与屏幕上显示对象的直接交互。
2. 减轻用户记忆负担。
 - (1) 减少用户的记忆负担。
 - (2) 设置有意义的默认值。
 - (3) 定义直观的快捷方式。
 - (4) 界面的视觉布局应基于真实世界的隐喻。
 - (5) 以渐进式方式披露信息。
3. 保持界面一致性。
 - (1) 让用户能将当前任务置于有意义的上下文中。
 - (2) 在同类应用程序中保持一致性。
 - (3) 如果过去的交互模型已形成用户预期，除非有充分理由，否则不要进行更改。

2. 软件的特征

- a. 软件是设计开发出来的一种逻辑的产品，与硬件产品有本质区别
- b. 软件产品质量的体现方式与硬件不同，软件不会“磨损”
- c. 软件因变更而逐渐退化。在软件的生命周期中，软件会经历变更。随着变更的实施，很可能会引入错误，使得故障率曲线出现峰值。在曲线能够恢复到原本的稳态故障率之前，又会有新的变更需求，导致曲线再次出现峰值。慢慢地，最低故障率水平开始上升。
- d. 软件产品的成本构成与硬件产品的成本构成不同。

软件产品的成本构成：人力成本、时间成本

硬件产品的成本构成：原材料成本

- e. 软件产品的失败曲线与硬件产品不同
- f. 大多数软件是根据顾客需求定制产生，硬件的可复用性比软件强很多

3. Describe the differences between blackbox testing and white-box testing. 描述黑盒测试和白盒测试之间的区别。

维度	黑盒测试 (Black-Box Testing)	白盒测试 (White-Box Testing)
核心视角	外部行为（用户可见功能）	内部结构（代码逻辑、路径）
测试对象	系统/模块的输入输出行为	程序代码、控制流、数据结构
目标	验证功能是否符合需求文档	验证代码逻辑是否正确
测试方法	等价类划分、边界值分析	路径覆盖
测试阶段	系统测试、验收测试	单元测试、集成测试

补充

1. 集成测试的三个策略



1. 增量集成 (Incremental Integration): 主张以小增量的方式逐步构造和测试程序，以便于发现和修正错误。

2. 自顶向下集成 (Top-down Integration):

- **描述:** 这是一种增量方法，从主控模块（主程序）开始，沿着程序的控制层次逐步向下集成和测试。它需要使用“桩模块”（stub）来代替尚未集成的低层模块。
- **具体方式:** 可以采用深度优先或广度优先的方式进行集成。

3. 自底向上集成 (Bottom-up Integration):

- **描述:** 这种方法从程序结构的最底层构件（原子模块）开始，将它们组合成完成特定功能的“簇”（build），然后向上进行构造和测试。它需要编写“驱动模块”（driver）来测试这些簇，但不需要桩模块。

4. 冒烟测试 (Smoke Testing):

- **描述:** 这是一种常用的集成测试方法，尤其适用于时间紧迫的项目。它通过频繁地（通常是每天）将软件构件集成到一个“构建”（build）中，并运行一系列测试来暴露那些可能导致项目延迟的严重错误（业务阻塞）。

5. 组合方法 (Combination Method / 三明治测试):

- **描述:** 这是一种被认为是“最好的折衷”的策略。它结合了自顶向下和自底向上的方法，即用自顶向下方法测试程序结构的较高层，用自底向上方法测试其从属层。

2. 验证与确认(Verification and Validation,V&V)

维度	验证 (Verification)	确认 (Validation) ≈ Acceptance Test	Acceptance Test
核心问题	我们在正确地构造产品吗?	我们在构造正确的产品吗?	客户愿意为产品付款吗?
定义	确保软件 正确地实现 某一特定功能的一系列活动	确保开发的软件可 追溯到客户需求 的另外一系列活动	由客户确认软件解决其业务问题
焦点	是否符合规格说明（设计文档、需求文档）	是否满足用户实际需求（真实场景）	
典型活动	代码评审、单元测试、静态分析	用户验收测试（UAT）、场景测试、Beta测试	
测试策略	自底向上集成（验证模块协作是否符合设计）	端到端系统测试（验证整体业务流程）	
测试方法	白盒测试（如路径覆盖）、代码检查	黑盒测试（如用户场景模拟）、可用性测试	
测试用例源			

	需求文档 → 设计文档 → 技术指标	用户故事 → 实际使用场景 → 业务目标	
--	--------------------	----------------------	--

3. 软件危机

- a. 软件危机是**软件开发能力落后于硬件发展与用户需求**所导致的**系统性失效状态**。其终结标志是软件工程学科的成熟与实践标准化。
- b. 表现为软件**开发效率低下、质量失控、成本暴增和交付延期**的恶性循环。

4. 软件生命周期

软件生命周期是软件产品从**概念探索到退役的全过程**，包括需求、设计、实现、测试、部署、运行和维护等阶段，直至软件不再使用。

5. 体系结构风格

 **以数据为中心的体系结构**（Data-centered architecture，黑板模式）

- 容易添加新的客户端软件
- 修改数据结构容易导致各个客户端都受影响

数据流的体系结构（Data-flow,管道-过滤器模式）

- 每一个处理步骤都封装在一个过滤器内，要处理的数据通过管道传递，**各过滤器无需了解其他过滤器的工作**

调用和返回体系结构,Call and return（**主程序-子程序体系结构**是调用-返回模式的典型代表）

- 该体系结构风格能够设计出一个相对易于修改和扩展的程序结构。

层次体系结构Layered（多层次体系架构模式）

N层架构

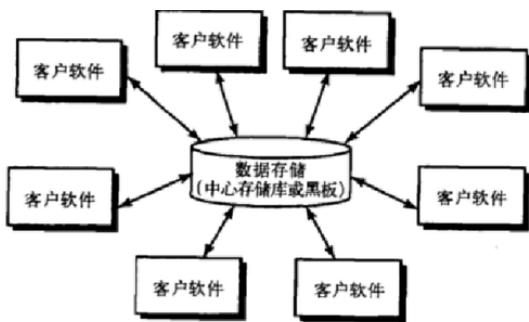


图9-1 以数据为中心的体系结构

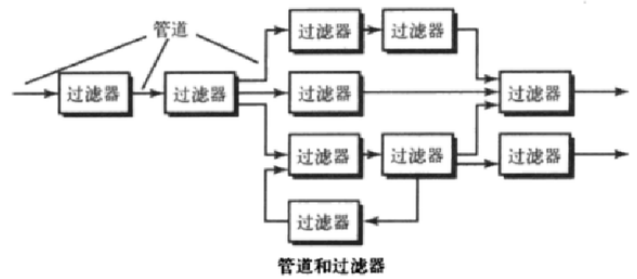


图9-2 数据流体系结构

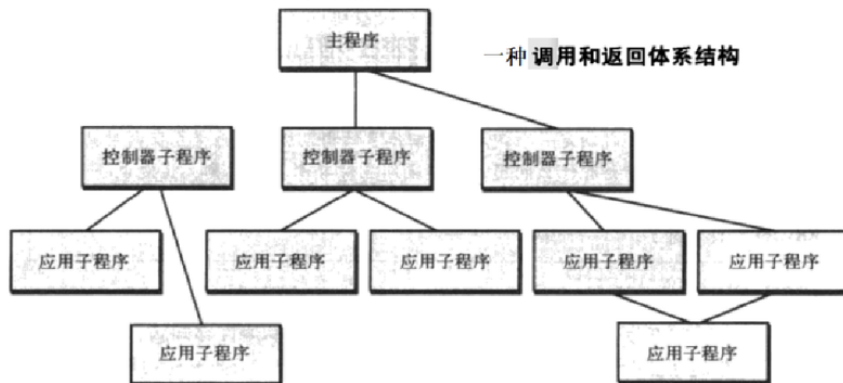


图9-3 主程序/子程序体系结构

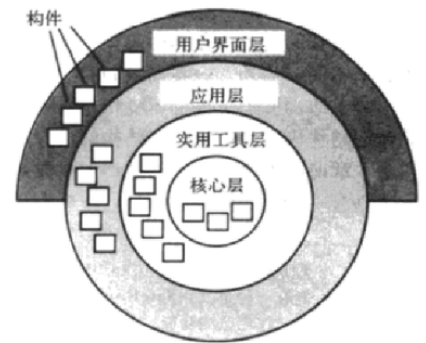


图9-4 层次体系结构

教材中对“面向对象体系结构”只做了文字描述，如下：

面向对象体系结构。系统的构件封装了数据和必须用于控制该数据的操作，构件间通过信息传递进行通信与合作。

补充***

一，设计概念（抽象Abstraction，求精Refinement，体系结构Architecture，关注点分离Separation of Concerns，模块化Modularity，信息隐藏Information Hiding，功能独立Functional Independence，方面Aspects，重构Refactoring，面向对象的设计概念Object-Oriented Design，设计类Design Classes，模式Pattern.....）

2，Black-Box Testing、White-Box Testing and Grey-box Testing；

3，Using your own words, describe the difference between verification and validation. Do both make use of test-case design methods and testing strategies.

4，Regression testing

5，PairTesting and pair programming

6，cyclomatic complexity

7，What are characteristics of testable software?

8，Testing should be performed by an independent test group

9，. List some problems that might be associated with the creation of an independent test group. Are an ITG and an SQA group made up of the same people?

内聚Cohesion，为组件的“专一性”。在面向对象系统的组件级设计语境中，内聚意味着一个组件或类**仅封装彼此密切相关且与该类或组件自身密切相关的属性和操作**。具有功能内聚、层次内聚和通信内聚的类和组件相对易于实现、测试和维护。应尽可能努力实现这些级别的内聚。

- 功能内聚。主要通过操作来体现，当一个模块完成一组且只有一组操作并返回结果时，就称此模块是功能内聚的。
- 分层内聚。由包、构件和类来体现。高层能够访问低层的服务，但低层不能访问高层的服务。
- 通信内聚。访问相同数据的所有操作被定义在一个类中。一般说来，这些类只着眼于数据的查询、访问和存储。

耦合性Coupling是对类之间相互连接/依赖程度的定性度量。随着类（和组件）之间的相互依赖程度增加，耦合度也会提高。在构件级设计中，一个重要的目标就是尽可能保持低耦合。耦合分类。

- 内容耦合。发生在当一个构件“暗中修改其他构件的内部数据”时。这违反了基本设计概念当中的信息隐蔽原则。
- 控制耦合。发生在当操作A调用操作B，并且向B传递了一个控制标记时。接着，控制标记将会指引B中的逻辑流程。这种耦合形式的主要问题在于，B中的一个不相关变更往往能够导致A所传递控制标记的意义也必须发生变更。如果忽略这个问题，就会引起错误。
- 外部耦合。发生在当一个构件和基础设施构件（例如，操作系统功能、数据库容量、无线通信功能等）进行通信和协作时。尽管这种类型的耦合是必要的，但是在一个系统中应该尽量将这种耦合限制在少量的构件或者类范围内。

-: Describe the differences between the software engineering terms **coupling** and **cohesion**? 描述软件工程术语耦合和内聚之间的区别？



模块化Modularity

1. 关注点分离表明**任何复杂问题如果被分解为可以独立解决和优化的若干块，该复杂问题能够更容易地被处理**。一个关注点是一个**特征或行为**，被指定为**软件需求模型的一部分**。通过将关注点分割为更小的关注点，使得解决一个问题需要付出更少的工作量和时间。
2. 模块化是关注点分离最常见的表现。软件被划分为独立命名的、可处理的构件，有时被称为模块，把这些构件集成到一起可以满足问题的需求。
3. Explain how effective **modular design** is achieved through functional independence of the individual modules? 解释如何通过各个模块的功能独立性来实现有效的**模块化设计**？

功能独立性通过高内聚（模块专注单一功能）和低耦合（模块间接口简洁）实现有效模块化设计：

- a. 高内聚确保模块内部自治，降低复杂度；
- b. 低耦合隔离变更影响，支持并行开发；
- c. 二者平衡可逼近最优模块数，最小化总成本；
- d. 最终效果是提升可维护性、测试性及工程管理效率。

 重构Refactoring是一个设计过程：不改变代码的**外部行为**而是改进其**内部结构**，旨在**优化构件内部结构和运行模式**

重构的基本步骤

1. 基于需求a，建立一个设计方案A
2. 基于需求a和相关需求a'，评估这个设计方案A，发现设计问题——软件结构问题或运作方式问题
3. 运用软件设计的一般性原则和方法（方面、OCP、LSP、DIP、ISP）解决这个设计问题，得到设计方案A'

前年考题

评阅教师	得分

一、简答论述题（本大题共3小题，共35分）。

1. There are five activities in generic process framework. Please apply them to describe the spiral process model. (10 Points)
2. List and describe three golden rules that should be applied when building any user interface. (9 Points)
3. Please describe the differences between following items briefly. (16 Points)
 - (1) Requirement and Specification;
 - (2) Black-Box Testing and White-Box Testing;
 - (3) Verification and Validation;
 - (4) Coupling and Cohesion;

作业剩下的题

1. Many modern applications change frequently before they are presented to the end user and then after the first versions have been used. Think about a few ways to build software to stop **deterioration** due to change. 构建软件以阻止因变更而导致恶化的几种方法。



- **采用敏捷开发方法：**敏捷开发强调快速迭代和灵活响应变化，通过将软件开发过程划分为多个短周期的迭代，以在每个迭代结束时根据用户的反馈和新的需求对软件进行调整和优化，从而更好地适应变化，防止软件因变化而退化。例如，Scrum 通过每日站会、迭代评审和回顾会议等方式，确保团队能够及时沟通和调整开发方向。
- **使用模块化设计：**将软件系统分解为多个独立的模块，每个模块负责特定的功能。模块之间通过清晰的接口进行交互。这样，当某个模块需要根据变化进行修改时，不会对其他模块产生太大的影响。
- **进行代码重构：**代码重构是指在不改变软件功能的前提下，对代码的结构进行优化和改进。通过代码重构，可以提高代码的可读性、可维护性和可扩展性，使软件更容易适应变化。定期进行代码重构可以确保软件代码的质量，降低因变化而导致的退化风险。
- **建立自动化测试：**通过编写自动化测试用例，可以在每次代码提交后自动运行测试，及时发现因变化而引入的错误，降低因变化而导致的退化风险。

2. What are the primary advantages of the component-based process model for software engineering? 基于组件的软件工程过程模型的主要优势



1. **软件复用性**减少“重复造轮子”的工作量，显著提升**开发效率**。
2. 通过复用现有构件，减少从零开发所需的人力、时间资源，**降低开发成本，缩短开发周期**
3. **支持迭代式演进**，适应需求变化，分阶段交付增强功能。模型本质是演化式的（类似螺旋模型），允许通过迭代逐步完善系统，每轮迭代可集成新构件或优化现有构件组合。
4. **标准化与质量保障**。商业构件通常经过严格测试，通过良定义的接口提供功能，需通过充分测试验证集成功能，降低模块集成风险，提升系统整体稳定性。

3. Which UML (unified modeling language) diagrams are useful in **scenario-based modeling**?



1. 用例图 (Use Case Diagram)

- 核心用途：可视化系统功能（用例）与参与者的交互关系，直观展示系统边界和功能范围。

2. 活动图 (Activity Diagram)

- 核心用途：描述用例中的操作流程与控制流，配合用例细化主场景和次场景。清晰呈现多路径交互，分解复杂场景的逻辑。

3. 泳道图 (Swimlane Diagram)

- 核心用途：划分不同参与者或系统组件的责任边界，明确跨角色的协作流程，解决职责交叉问题

4. Describe the roles of the three sections of CRC(class responsibility collaborator) cards 描述 CRC（类-责任-协作者）卡片三个部分的作用



1. 类名 (Class)

- 明确类的名称，分类实体类型。
 - 实体类：直接映射业务实体，存储持久化数据。
 - 边界类：管理用户界面与系统交互。
 - 控制类：协调复杂操作与对象协作。

2. 职责 (Responsibility)

- 定义类的能力，描述类“知道什么”（属性）和“能做什么”（操作）。
- 分配原则（Wirfs-Brock五大准则）：
 - i. 智能均匀分布：避免集中少数“灵巧类”，提高可维护性。
 - ii. 抽象层级一致：同一类的职责需处于相同抽象层次（如不同时包含复杂计算与简单事务）。
 - iii. 封装行为与数据：操作数据的职责与数据本身属于同一类（如 `Sensor` 类负责读取自身状态）。
 - iv. 信息唯一归属：特定数据仅由一个类管理（避免分散）。
 - v. 共享职责：相关类可共享通用行为（如多个游戏角色类共享 `update()` 和 `display()`）。

3. 协作者 (Collaborator)

- 标识协作依赖：列出帮助完成职责的其他类。当类无法独立完成职责时，需请求协作者提供信息或执行操作。

- 协作关系类型：
 - 部分关系 (is-part-of)：谁是谁的一部分
 - 知识依赖 (has-knowledge-of)：需从获取状态信息。
 - 间接依赖 (depends-upon)：通过第三方类间接协作

5. List the characteristics that should be considered when considering **potential classes** for inclusion in an analysis model 列出在考虑将潜在类纳入分析模型时应考虑的特征
6. What are the steps needed to build a behavioral model? 构建行为模型需要哪些步骤？——如何从用例规格说明书转换为顺序图



1. 评估用例，理解交互顺序，输出每个用例的活动序列
2. 分析交互步骤，判断是否有交互信息，若有则识别为事件，关联其关联对象
3. 基于事件流构建顺序图，为用例生成事件序列
 - a. 画箭头，事件传递方向
 - b. 箭头上方写事件，并传递参数
4. 为每个类构建状态图
 - a. 建立状态节点：对象的主动状态
 - b. 转移箭头：事件触发的状态跳转
 - c. 守卫条件：转移需满足的布尔条件，写在箭头上方
 - d. 伴随动作：状态转移时执行的操作
5. 评审行为模型
 - a. 是否完整
 - b. 状态转移逻辑是否正确
 - c. 对比顺序图和状态图，是否一致
 - d. 一致则通过验证

7. Describe the principle of **information hiding** as it applies to software design. 描述信息隐藏在软件设计中的应用原则



建议模块应该“具有的特征是：**每个模块对其他所有模块都隐蔽自己的设计决策**”。换句话说，模块应该规定并设计成为在**模块中包含的信息(算法和数据)**不被不需要这些信息的**其他模块访问**。

效果：隐蔽意味着通过定义一系列独立的模块可以得到有效的模块化，独立模块相互之间只交流实现软件功能所必需的那些信息。抽象有助于定义构成软件的过程(或信息)实体。隐蔽定义并加强了对模块内过程细节的访问约束和对模块所使用的任何局部数据结构的访问约束。

8. What are the elements that make up a software architectural style? 构成软件架构风格的要素有哪些？



- (1)完成系统需要的某种功能的一组**构件**(例如，数据库、计算模块)；
- (2)能使构件间实现“通信、合作和协调”的一组**连接件**；
- (3)定义构件如何集成为系统的**约束**；
- (4)**语义模型**，能使设计者通过分析系统组成成分的已知属性来理解系统的整体性质

9. List three characteristics that can serve as a guide to evaluate design quality. 列出三个可作为评估设计质量的指导原则的特性。



1. 为改变而设计【设计应适应，或促进改变】
2. **关注点分离**【要单独关注问题的一个方面，而不是同时解决所有方面。】
3. **信息隐藏**【隐藏一个模块的实现细节来降低对软件系统其他部分的影响。信息隐藏是通过将一个类的数据成员私有化和保持类的接口的稳定性来实现的。】
4. **高内聚**【高内聚准则建议模块设计应该尽力达到模块的函数对于模块的核心功能的更高的相关。衡量由一组函数构成的一个模块，函数对于模块核心功能的相关程度】
5. **低耦合**【低耦合度量一个模块运行时行为对其他模块的运行时的影响的程度；也度量当一个给定的模块的实现发生改变时，有多少模块也得改】
6. **保持简单和直接**【KISS原则鼓励简单、直接、容易理解的设计，即一个对象在处理请求的时候不问问题，那么它淳朴；一个对象只知道如何做一件事情而别的事情都不知道，则它足够简单。】

10. How does the object-oriented view of component-level design differ from the traditional view? 面向对象的构件级设计观点与传统观点有何不同？



构件本质不同

- **传统设计：**
 - 构件为**“功能模块”**（函数/过程），分为控制模块（调度逻辑）、问题域模块（业务计算）和基础设施模块（技术支撑）。

- **数据与行为分离**：数据通过全局结构共享（如C的 `struct` ），模块通过参数传递操作数据，导致**高耦合**。

- **面向对象设计：**

- 构件为类（Class），封装数据（属性）与行为（方法）于一体。
- **数据私有化**：属性通常为 `private`，通过公共方法（如 `get/set`）访问，符合**封装原则**，降低耦合风险。

2. 交互机制不同

- **传统设计：**

- **显式函数调用**：模块通过参数传递数据（如 `calculate_tax(order)` ），依赖具体实现。
- 修改接口易引发连锁错误。

- **面向对象设计：**

- **消息传递（方法调用）**：对象通过公共方法协作（如 `order.calculate_tax()` ）。
- **依赖接口而非实现**（如 `PaymentProcessor` 依赖 `IPaymentGateway` 接口），符合**依赖倒置原则（DIP）**。

3. 扩展性与复用机制不同

- **传统设计：**

- 扩展需**修改现有函数**（如新增折扣需重写 `calculate_total()` ），违反开闭原则。
- 复用单元为**函数库**（如数学工具集），粒度细且组合复杂。

- **面向对象设计：**

- 扩展通过**继承**（子类覆写方法）或**组合**（动态替换构件），符合**开闭原则（OCP）**。
- 复用单元为**可复用类**（如 `Logger` 工具类），支持设计模式（如策略模式）。

11. What elements of a user interface design can be evaluated without building a work in a computer prototype? 在不构建工作计算机原型的情况下，可以评估用户交互界面设计的哪些元素？
12. 收集关于架构模式(Architectural Pattern) 的资料，简要介绍你感兴趣的3种架构模式，包括该模式的结构视图，优点和缺点分析等。

13. what are the attributes of a good software test? 一个好的软件测试有哪些属性?



1. 好的测试具有**较高的发现错误的可能性**。
2. 好的测试是**不冗余的**。测试时间和资源是有限的，执行与另一个测试有同样目标的测试是没有意义的。每个测试都应该有不同的目标(即使是细微的差别)。
3. 好的测试应该是**“最佳品种”**。在一组具有类似目的测试中，时间和资源的有限性可能只影响这些测试的一个子集的运行。在这种情况下，应该使用最有可能发现所有类别错误的测试。
4. 好的测试应该**既不太简单也不太复杂**。尽管将一系列测试连接为一个测试用例有时是可能的，但潜在的副作用会掩盖错误。

14. What are the key differences between validation testing goals and acceptance testing goals?

15. What is **equivalence partitioning** as it applies to software testing? 在软件测试中，什么是等价类划分?



等价类划分是一种**黑盒测试方法**，它将程序的输入划分为若干个数据类，从中生成测试用例。理想的测试用例可以单独发现一类错误，否则在观察到一般的错误之前需要运行许多测试用例。

16. Why is **regression testing** an important part of any **integration testing** procedure? 为什么回归测试是任何集成测试过程的重要组成部分?



每当**加入一个新模块**作为集成测试的一部分时，**软件发生变更**，建立了新的数据流路径，可能出现**新的I/O**，以及调用**新的控制逻辑**。这些变更可能会使原来可以正常工作的功能产生问题。

在集成测试策略的环境下，回归测试是**重新执行已测试过的某些子集**，以确保变更没有传播**不期望的副作用**。

17. Describe three **control structure testing strategies**. 描述三种控制结构测试策略。

18. Describe object-oriented unit testing. 描述面向对象单元测试。/How does unit testing differ for object-oriented testing as compared to conventional software unit testing? 与传统软件单元测试相比，面向对象测试的单元测试有何不同?

19. What is scenario-based testing? 什么是基于场景的测试?



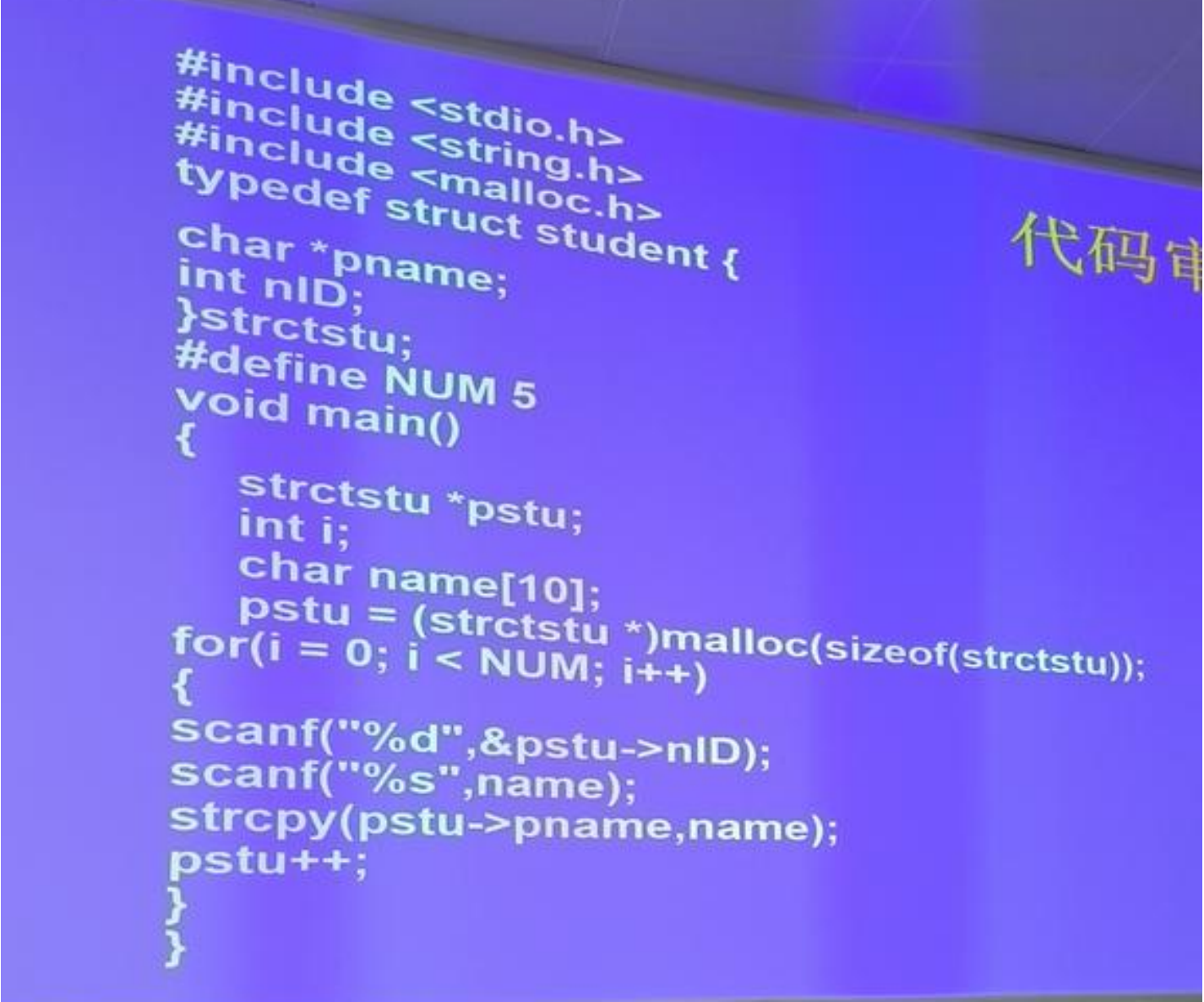
基于场景的测试**关心用户做什么**，而不是产品做什么。这意味着**捕获用户必须完成的任务**(通过用例)，然后在测试时使用它们及其变体。

场景可以发现交互错误。基于场景的测试倾向于用单一测试检查多个子系统。

基于场景的测试在面向对象系统的确认中占优势，使用例成为确认测试的主要驱动者。

20. Describe three partitioning strategies that can be used when performing class level testing for OO systems. 描述三种在对OO系统进行类级测试时可以使用的分区策略。

其他真题



```
#include <stdio.h>
#include <string.h>
#include <malloc.h>
typedef struct student {
    char *pname;
    int nID;
}strctstu;
#define NUM 5
void main()
{
    strctstu *pstu;
    int i;
    char name[10];
    pstu = (strctstu *)malloc(sizeof(strctstu));
    for(i = 0; i < NUM; i++)
    {
        scanf("%d",&pstu->nID);
        scanf("%s",name);
        strcpy(pstu->pname,name);
        pstu++;
    }
}
```

代码审查

分析设计 40'

今年必考

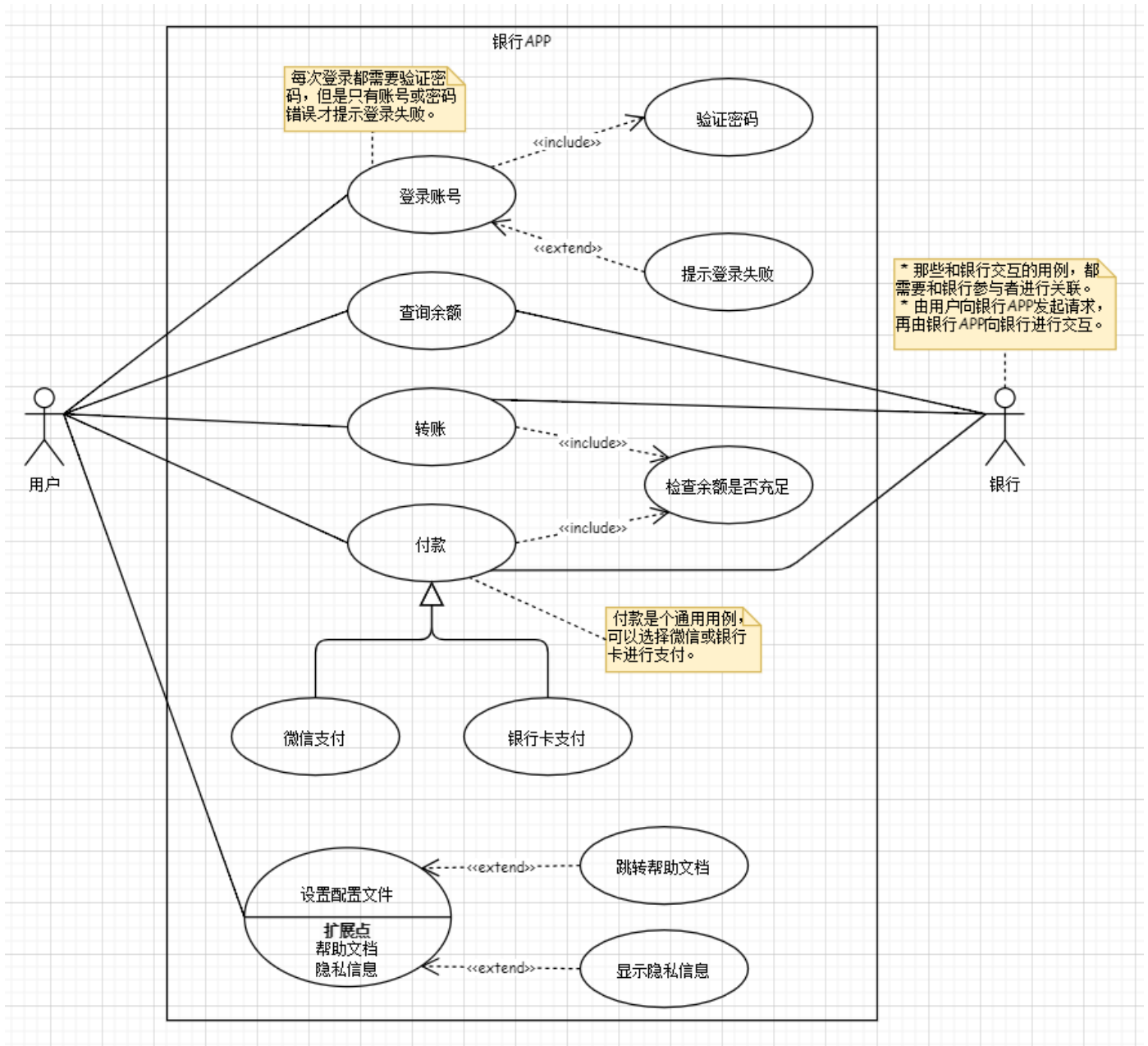
1. 用例图

方法

include关系就是做A的时候一定伴随做B

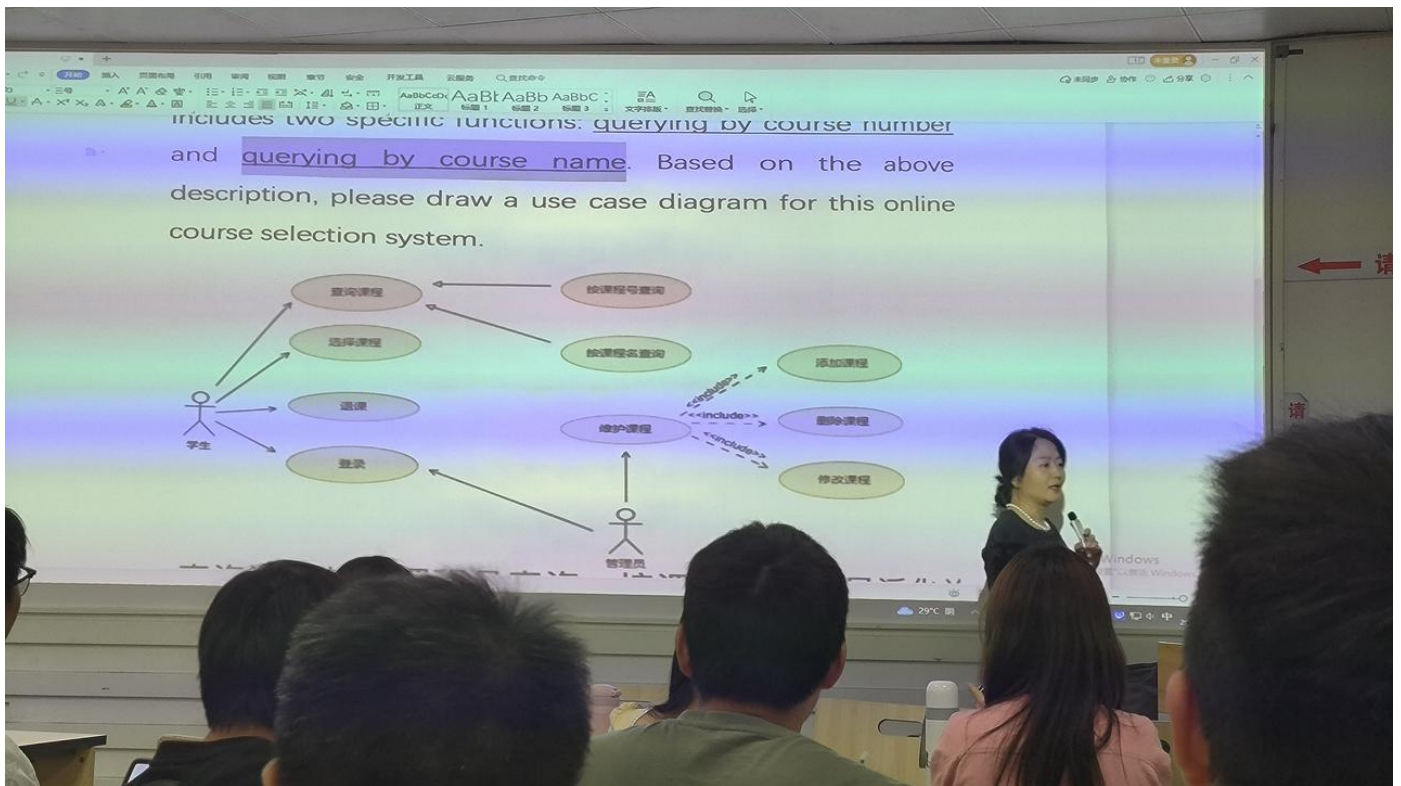
extend关系就是做A的时候不一定B，需要进一步触发B（密码输错、点击发送等）

泛化关系就是通过AB不同的方式实现C



例题

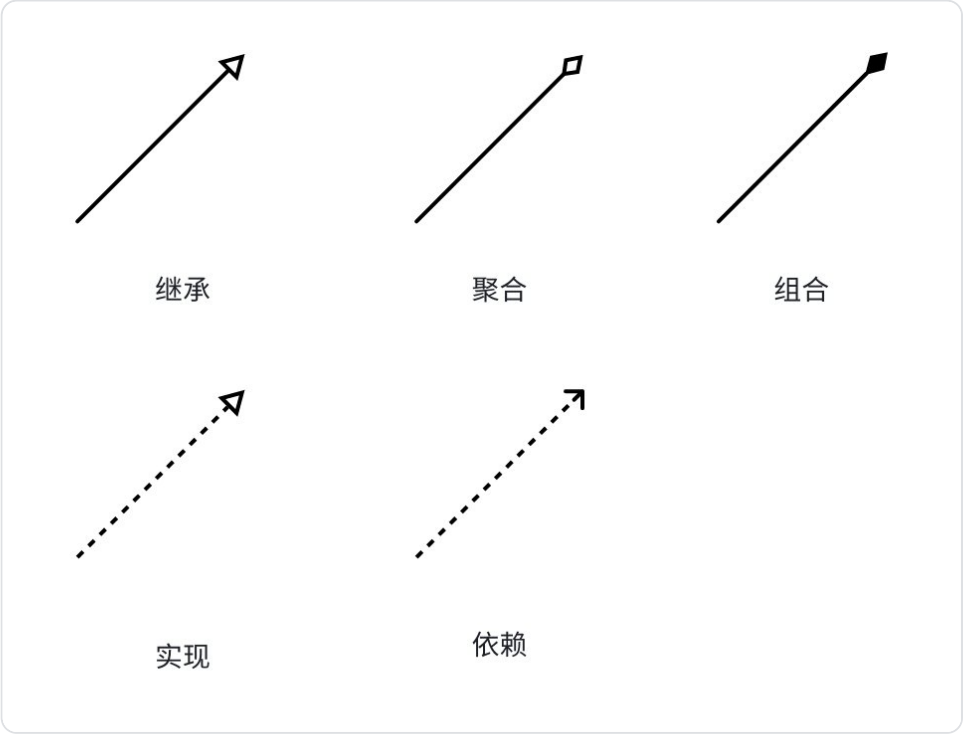
- a. In an online course selection system, students can query course information, select courses, drop selected courses and log in to the system. The system administrator can log in, has an extended function to retrieve passwords, and can maintain course information (such as adding, modifying, and deleting courses). Additionally, querying course information includes two specific functions: querying by course number and querying by course name. Based on the above, please draw a use case diagram for this online course selection system.



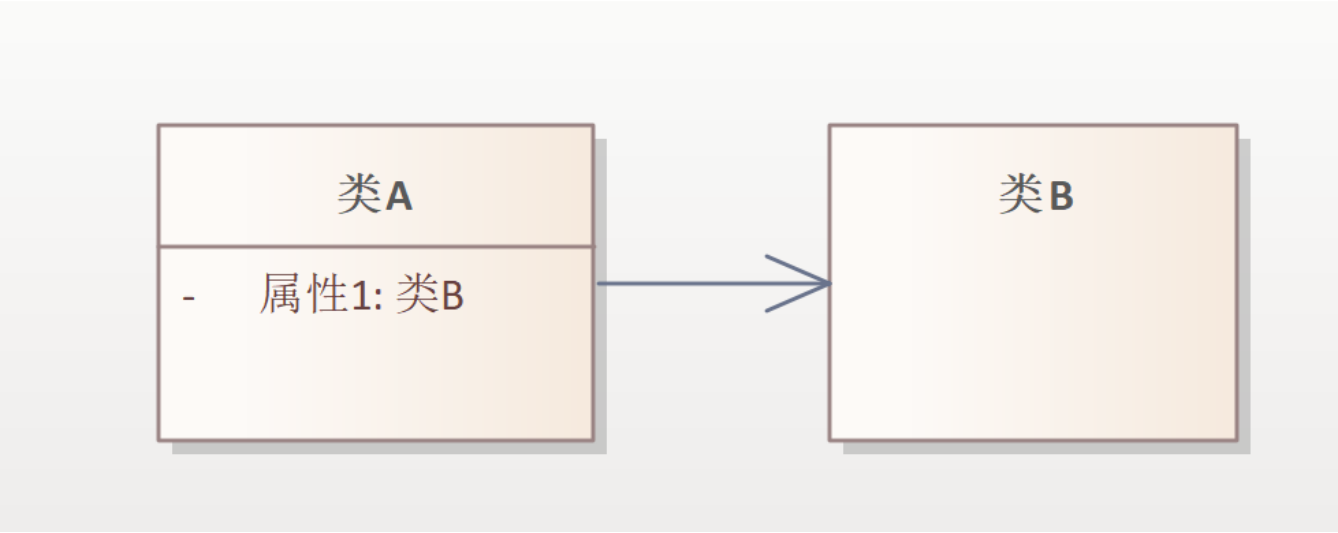
The functional requirements of the teacher's end of the "Teaching Resource Platform" are as follows. Teachers and students need to log in to the "Teaching Resources Platform" before they can use all the functions of the system. If you forget your password, you can retrieve it using the "Forgot Password" function. Teachers can edit course information. Teachers can publish courseware for their own courses. Teachers edit and assign homework online. Students can submit their electronic assignments on the platform. Teachers can grade assignments on the platform, that is, give scores to assignments. Please draw a use case diagram according to the above requirements.

The university library mainly provides services such as book borrowing for teachers and students. These services mainly include: book information management, borrowing, returning, reservation borrowing, etc. The system uses the campus card to identify readers and uses the campus card to process borrowing and returning books. There are different management requirements for different readers (teachers and students). For example, students can borrow up to 5 books, while teachers can borrow up to 10 books. If a reader has overdue books, they cannot borrow books. They need to pay the fine and return the books before they can continue to borrow books.

2. 分析类图

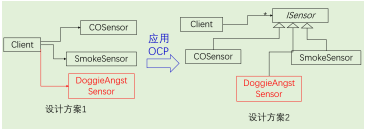


关联关系：类A→类B，则类A的属性类型中有类B即表示为



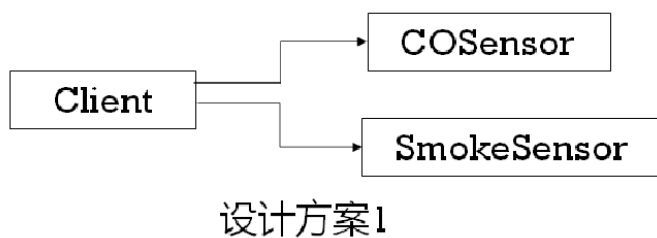
依赖关系：类A的操作中返回类型会有类B

方法

		举例违反	
OCP开闭原则	模块/构件应该对外延具有开放性，对修改具有封闭性	同类的东西没有归整出总类作为接口（car truck要归vehicle）	
	子类可以替换它们的基类		

LSP Liskov子类替换原则		不是同类的却归成了同一 老大不会飞的penguin和 会飞的swan都归为 bird(要求必须fly)	
DIP依赖倒置原则	依赖于抽象，而非具体实现。		
ISP接口分离原则	多个客户专用接口比一个通用接口要好。	只需要会coding的学生 学了一堆物理历史生物	

■ 违反OCP



```
//Client::Check方法
{
while(读SensorsConfigFile直至文件尾, 每次读出一个配置项目)
{
...//获取该配置项目的名称, 它表明了Sensor的类型type
switch(type)
case COSensor
{
COSensor s1 = (COSensor)Class.forName(classname).newInstance();
s1.checkk( );
}
case SmokeSensor
{
SmokeSensor s2 = (SmokeSensor)Class.forName(classname).newInstance();
s2.checkk( );
}
...//其它case
}
}
```

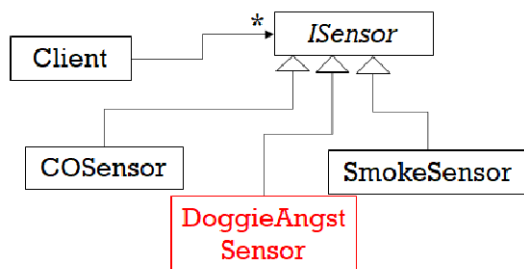
实现了: Client需要check各个Sensor

```
//Client::Check方法
{
while(读SensorsConfigFile直至文件尾, 每次读出一个配置项目)
{
...//获取该配置项目的名称, 它表明了Sensor的类型type
switch(type)
case COSensor
{
COSensor s1 = (COSensor)Class.forName(classname).newInstance();
s1.checkk( );
}
case SmokeSensor
{
SmokeSensor s2 = (SmokeSensor)Class.forName(classname).newInstance();
s2.checkk( );
}
case DoggieAngstSensor
{
DoggieAngstSensor sx= (DoggieAngstSensor)Class.forName(classname).newInstance();
sx.checkk( );
}
...//其它case
}
}
```

修改了
内部逻辑
违背了OCP

如何修改?

新建一个接口, 让他们实现, client依赖。=>解耦switch-case



设计方案2

```

//Client::Check方法
{
  while(读SensorsConfigFile直至文件尾, 每次读出一个配置项目)
  {
    ...//获取该配置项目的名称, 它表明了Sensor类名称classname
    ISensor s = (ISensor)Class.forName(classname).newInstance();
    sensors.add(s); //sensors是sensor的集合
  }
  for(ISensor s:sensors)
  {
    s.check();
  }
}

```

//Client::Check方法

当需求进化时, 设计方案2不需要改变Client::Check的逻辑

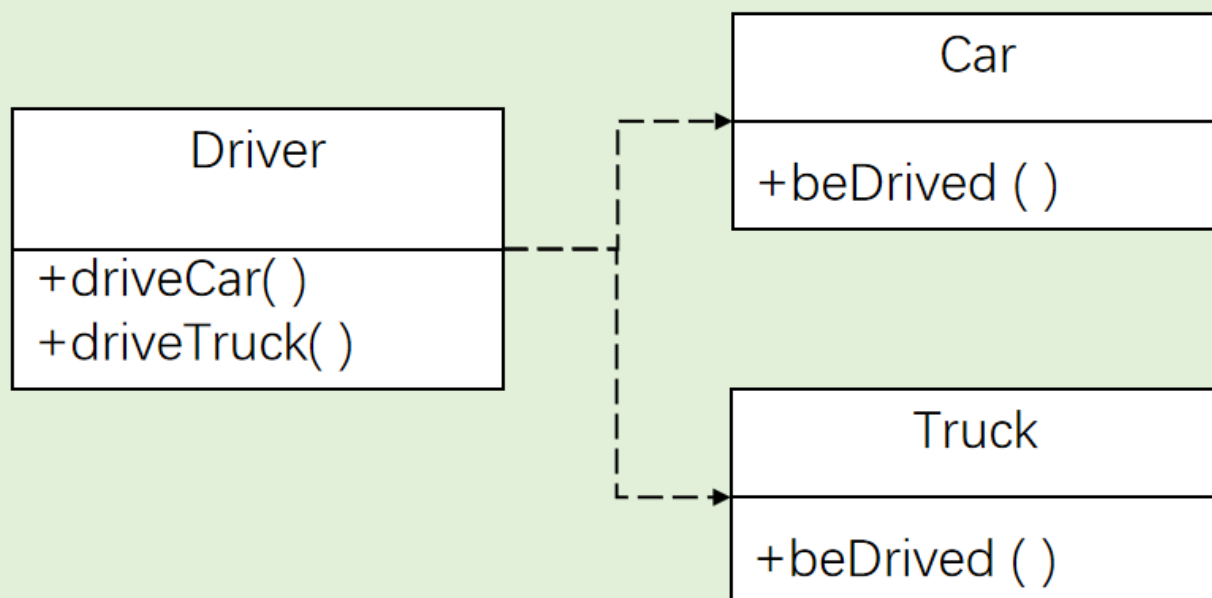
```

{
  while(读SensorsConfigFile直至文件尾, 每次读出一个配置项目)
  {
    ...//获取该配置项目的名称, 它表明了Sensor类名称classname
    ISensor s = (ISensor)Class.forName(classname).newInstance();
    sensors.add(s); //sensors是sensor的集合
  }
  for(ISensor s:sensors)
  {
    s.check();
  }
}

```

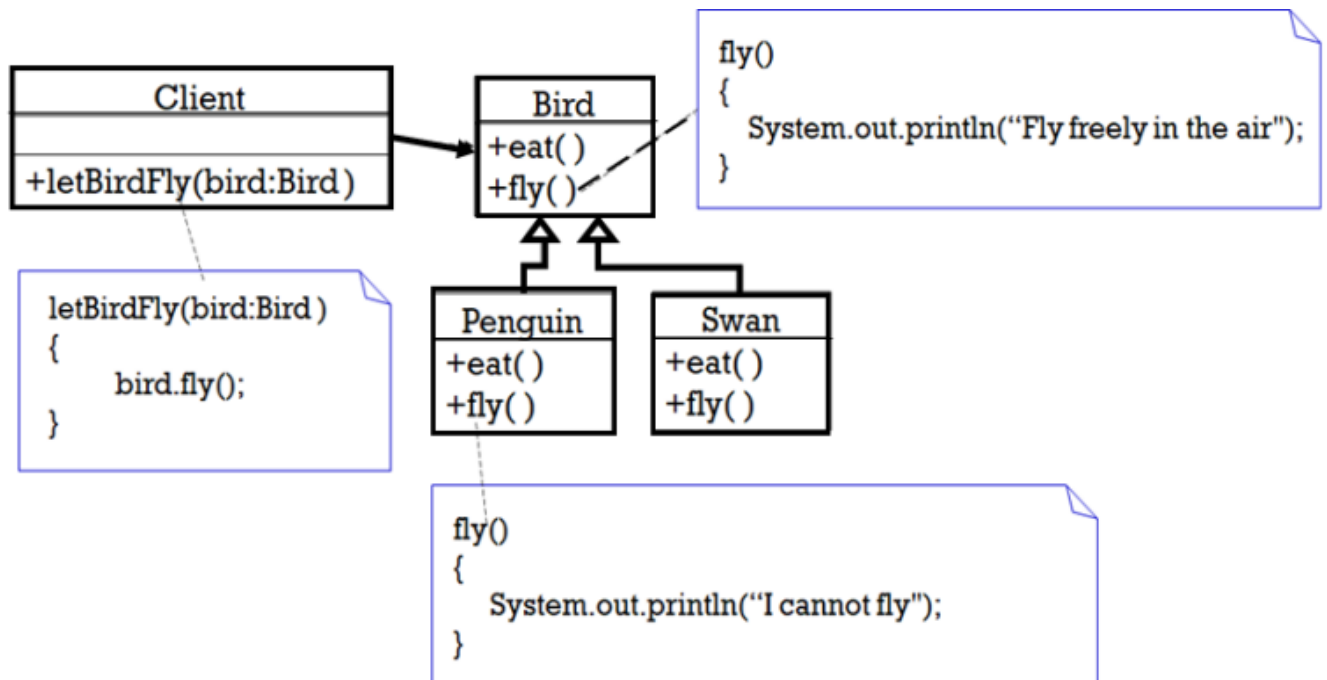
内部逻辑稳定, 通过接口向Client聚合

练习



■ 违反LSP

A component-level design is given in the following class diagram. Please assess whether this design is following the basic design principles or not. If not, please give your comments as well as improvement solution.



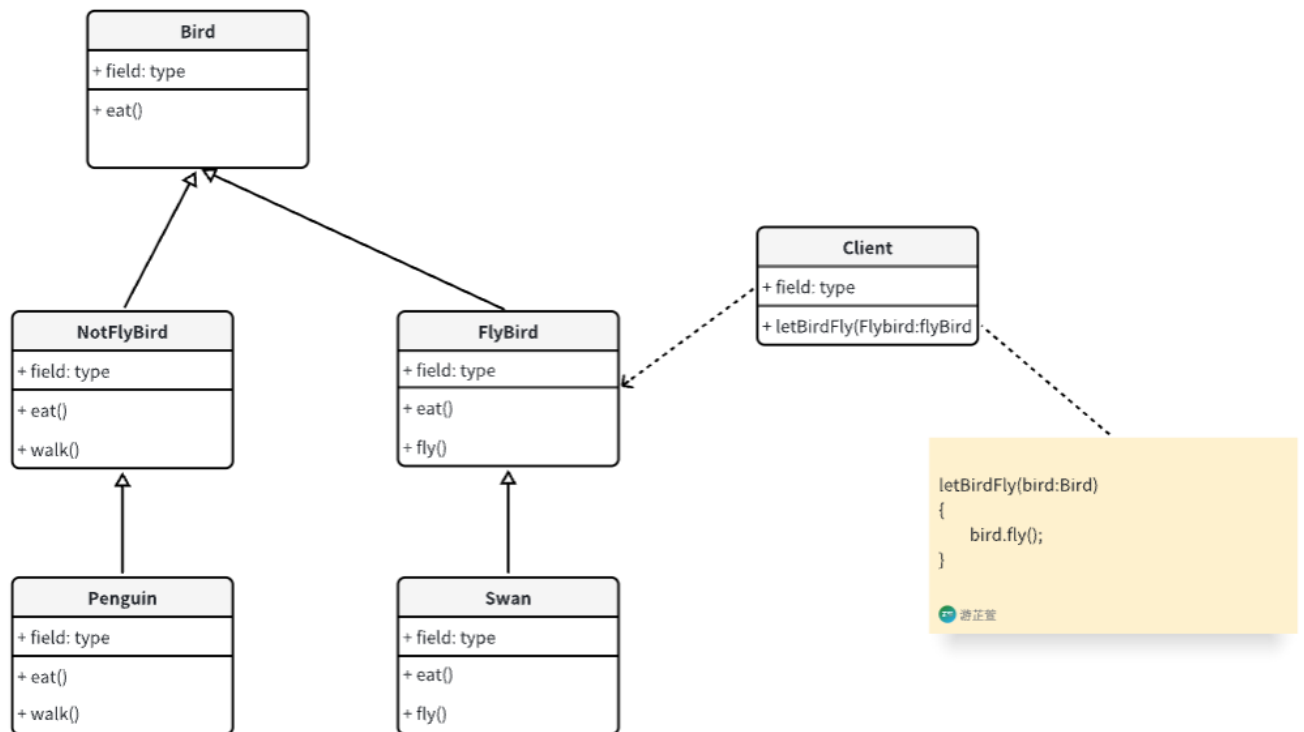
Tips: This component is composed of Client, Bird, Penguin and Swan. When executing the method `letbirdfly (Bird)` of Client, it is expected to output the statement "fly free in the air". However, when replacing the base class Bird with subclass Penguin, the output statement "I cannot fly" is inconsistent with the expectation.

如何修改？

A和B有相同有不同，却都继承于C

则新建抽象类DW，只有A有的放进D，A继承D，AB都有的放进W，CD继承W

=>解耦if-else



■ 违反DIP

The code below is an example of component design that violated the principles of DIP (Dependency Inversion Principle). Please explain why and design the correct UML class diagram with Wechat Pay in order to obey DIP principles.

代码块

```

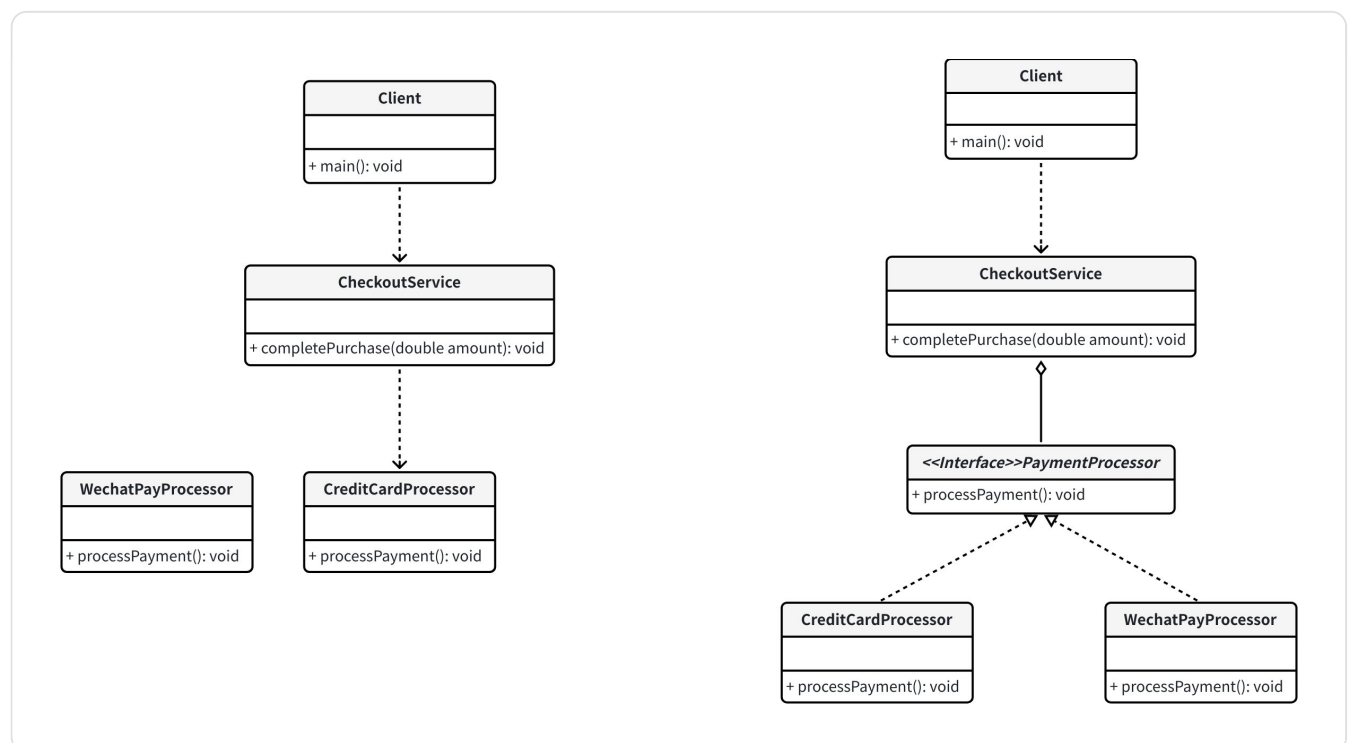
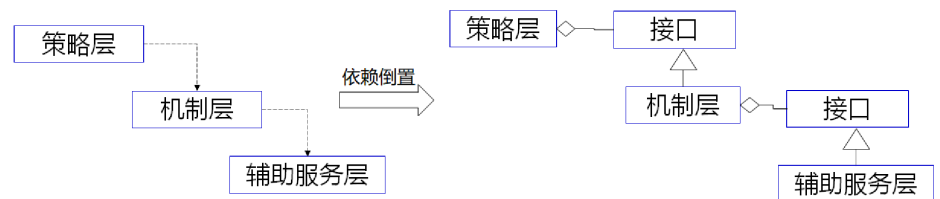
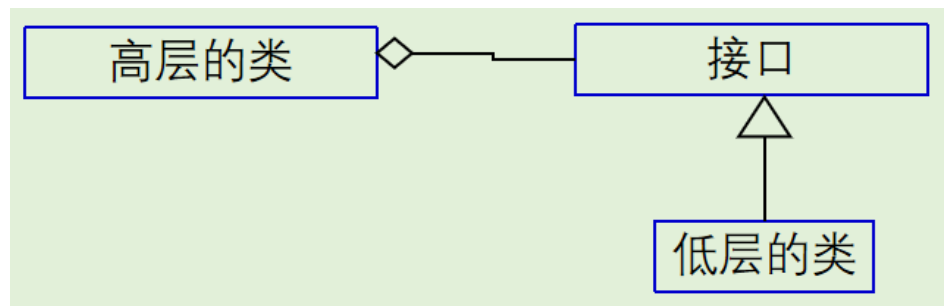
1  class CreditCardProcessor {
2      public void processPayment(double amount)
3      {System.out.println("Processing credit card payment of $" + amount);
4      }
5  }
6  class CheckoutService {
7      private CreditCardProcessor processor = new CreditCardProcessor0;
8      public void completePurchase(double amount){
9          processor.processPayment(amount);
10     }
11 }
12 public class Main {
13     public static void main(String[] args){
14         CheckoutService checkout= new CheckoutService();
15         checkout.completePurchase(150.0):
  
```

Tips: Developers are trying to integrate WeChat Pay into existing payment systems. However, they are finding that each time they add a new payment method, they need to modify the CheckoutService code to accommodate the new payment.

如何修改？

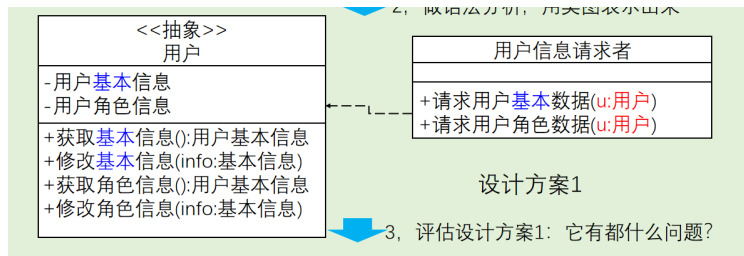
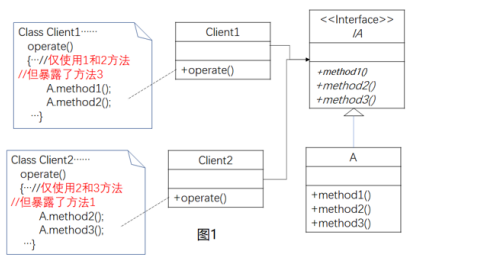
高层中的类可以聚集一个接口/抽象类，

低层的类依赖于（实现）这个接口/抽象类。



■ 违反ISP

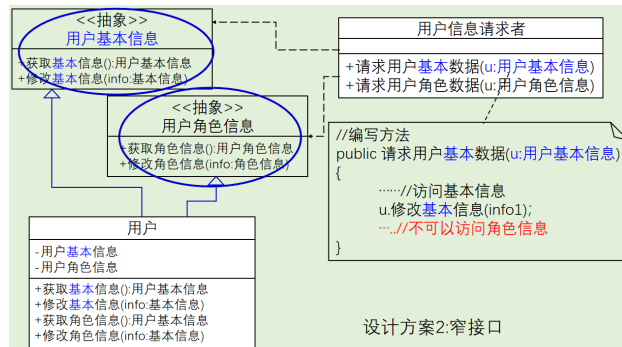
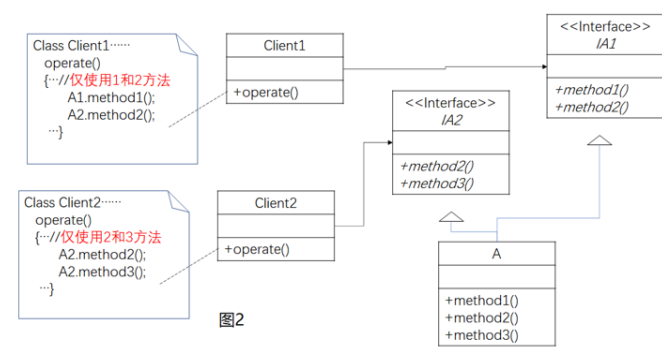
多个用户使用接口但是却不使用（浪费）或不允许使用所有方法（宽接口）



如何修改？

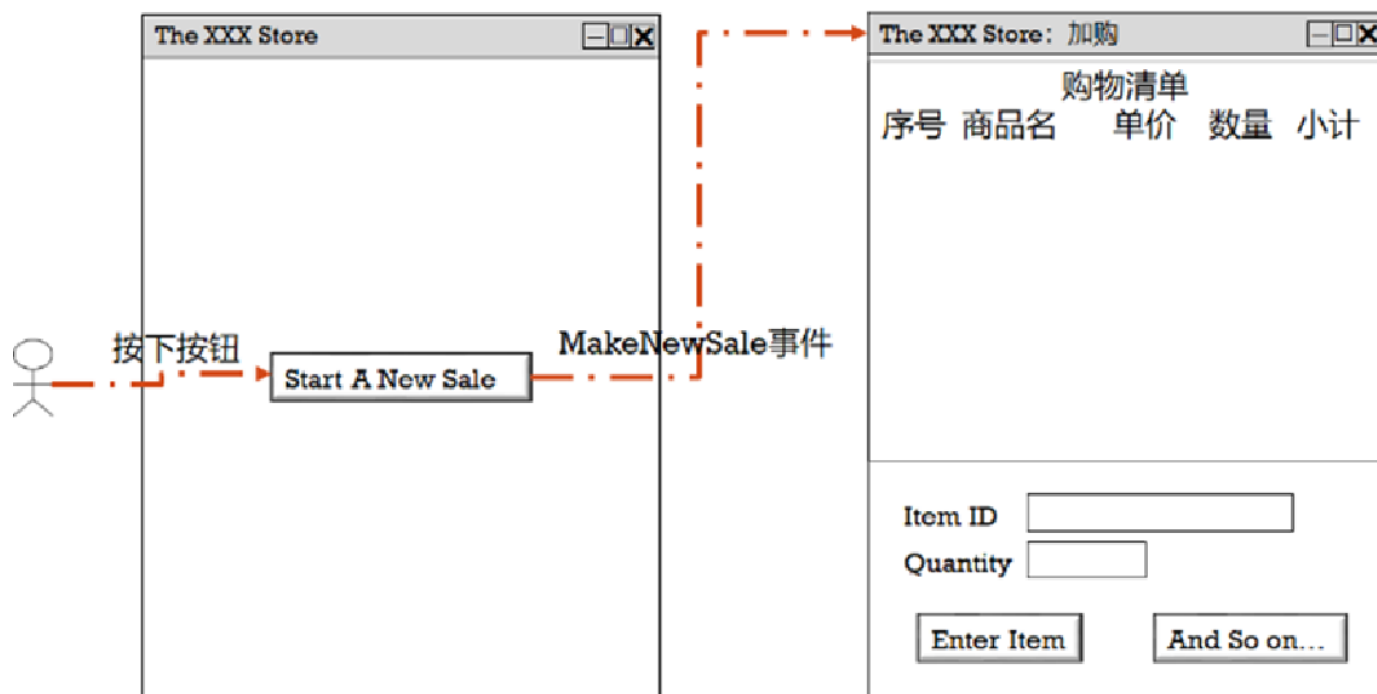
删除通用接口，为每个主要的客户类型都设计一个特定的接口（抽象类也类似的做法）

让一个实现类实现所有接口



3. 类图+界面设计

输入输出的这些数据要标识清楚，比如什么怎么样方便



4. Carefully read the material:

Material:

Students can search for Program through the "Overseas Project Application System". If he find a project of interest, he can continue to view Program Details. If the project details can attract him, he can submit the application online. Each student can apply for up to two projects.

(1) Identify the classes and relationships between them, and use UML class diagrams to represent these classes and relationships. (8 Points)

(2) Please design a GUI sequence for the "Overseas Project Application System". Please provide the layout of each GUI and the navigation relationships between the GUI (GUI=Graphical User Interface). (8 Points)

4. 测试图

Exercise

There is a program to calculate the price of freight.
Given pricing rules of freight (货运定价规则) as following:

The package charges are as follows: If the receiving place is within the province and the weight is less than or equal to 25kg, 8 yuan per kilogram for express mail and 4 yuan per kilogram for slow mail. If the receiving place is in another province and the weight is less than or equal to 25kg, the express mail is 12 yuan per kilogram and the slow mail is 8 yuan per kilogram. If the weight is more than 25kg, the overweight part will be charged 2 yuan per kilogram

- (1) Draw a Flow Graph with simple condition for this program.
- (2) Compute Cyclomatic complexity of the program.
- (3) List a set of independent path for conducting basic path testing.
- (4) Design test cases for conducting basic path testing.

3. Given the program logic as follows: (14 Points)

```
public int binSearch(int arr[], int objectValue) {  
    int left = 0;  
    int right = arr.length - 1;  
    int mid;  
    while (left < right) {  
        mid = (right - left) / 2 + left;  
        if (arr[mid] == objectValue) {  
            return mid;  
        } else if (objectValue < arr[mid]) {  
            right = mid - 1;  
        } else {  
            left = mid + 1;  
        }  
    }  
    return -1;  
}
```

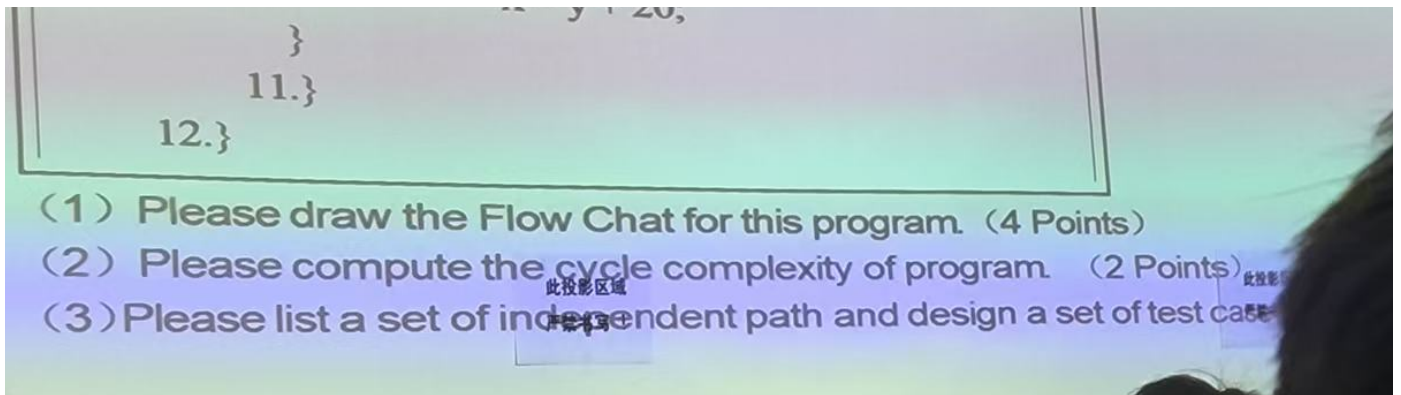
(1) Please draw the Flow Chart for this program. (5 Points)

(2) Please compute the cycle complexity of program. (3 Points)

(3) Please list a set of independent path and design a set of test case for basic path testing. (6 Points)

3. Given the bubbleSort logic as follows: (10 Points)

```
void Sort(int iRecordNum, int iType)  
{  
    1. int x = 0;  
    2. int y = 0;  
    3. while (iRecordNum-- > 0) {  
    4.     if (0 == iType) {  
    5.         x = y + 2;  
    6.         break;  
    7.     }  
    8.     else if (1 == iType) {  
    9.         x = y + 10;  
    10.    }  
    11.    else {  
    12.        x = y + 20;  
    13.    }  
    14. }
```

其他图

ER图

评阅教师	得分

三、应用、设计及分析题（本大题共 4 小题，共 50 分）。

1. A news management system is mainly used to publish news, the administrator only needs one, a logging in, you can Post news in the background. Anyone can browse the news, and the viewer (register as a member of the system, and can comment on the news after registration. Administrators (manage news, comments, and registered members in the background, such as modifying and deleting.

(1) Please develop an entity-relationship diagram that describes data objects, relationships, and attributes. (4 Points)

(2) Please draw a use case diagram of the news management system. (6 Points)

4. You have been asked to build a network-based course registration system for your university. Please describe analysis models of this system with E-R Diagram and Activity Diagram. (10 points)

任课教师: _____ 学号: _____ 姓名: _____

活动图

状态图

时序图

非标 2*10 ‘

1. 软件工程是什么？ what is soft engineering in your opinion?

软件工程：（1）将系统的、规范的、可量化的方法应用于软件的开发、运营和维护；即把工程学应用于软件。（2）对（1）中所述方法的研究。

2. 2022-2023: please describe the Agile process model in your own words and explain why it is important for software processes to be agile? (List the key issues stressed by an agile philosophy of software engineering 列出软件工程敏捷理念强调的关键问题)



敏捷开发是一种灵活、机动、有适应能力和精益的软件工程理念。它并非指某一个单一的方法，而是一系列过程模型的总称。

软件工程的敏捷理念**强调4个关键问题**：自我组织团队对所开展工作具有控制力的重要性；团队成员之间以及开发参与者与客户之间的交流与合作；对“变更代表机遇”的认识；以及强调快速软件交付以让客户满意。

敏捷开发原因：

1. **敏捷开发为了响应现代经济环境的高度不确定性** 在现代经济环境中，市场状况瞬息万变，客户和终端用户的需求不断演变，新的竞争威胁会毫无征兆地出现。从业者必须以一种能让他们保持敏捷性的方式来开展软件工作，以适应现代业务的需求。
2. **敏捷开发为了降低变更带来的高昂代价** 上述的不确定性必然导致变更，而变更会付出昂贵的成本。敏捷开发能够通过软件过程来降低由变更所引起的代价。
3. **敏捷开发为了更好地适应“人”的特点和弱点** 传统的过程模型一大缺陷是“忘记了开发计算机软件的人员的弱点”。软件工程师不是机器人，不能始终一致地做同一件事是人性的弱点，因而传统软件工程的高度纪律性的方法学非常脆弱。敏捷开发承认并试图利用更“宽容”的方式来适应人的天性，而不是强制施加脆弱的纪律。
4. **敏捷开发本质上为了克服传统软件工程的弱点** 它并非要完全抛弃传统软件工程，而是对其进行改进和发展，以适应新的挑战。

敏捷过程模型能解决上述这些问题。主要方法包括：

- **极限编程 (XP)**：应用最广泛的模型之一。它围绕**策划、设计、编码和测试**四个活动组织，通过一系列技术创建频繁的软件发布。

1. 策划：

开始于倾听产生一系列“用户故事”（需求获取活动），描述即将建立的软件需要的输出、特征以及功能。技术成员**理解软件的商业背景**以及**充分感受要求的输出\主要特征\主要功能**。

每个故事由客户书写在一张**索引卡**上，客户根据对应特征或功能的综合业务价值**标明优先级**。技术人员评估每一个故事的**开发时间成本**。如果某个故事的成本**超过了3个开发周**，则将请客户把该故事**进一步细分**，重新赋予权值并计算成本。**新故事可以在任何时刻书写**。

2. 设计

遵循**KIS原则**，使用简单而不是复杂的表述。设计为故事提供**不多也不少**的实现，**不鼓励额外功能性**设计。

如果在某个故事设计中碰到困难，推荐立即建立这部分设计的**可执行原型**，实现并评估设计原型，从而降低风险。

3. 编码

开发一系列用于检测本次（软件增量）发布的包括所有故事的单元测试，

一旦**建立单元测试**，开发者集中精力于**必须实现的内容以通过单元测试**。**不需要加任何额外的东西(KIS，保持简洁)**。

一旦编码完成，就可以**立即完成单元测试**，从而向开发者提供即时的反馈。

ps：结对编程。建议**两个人面对同一台计算机共同为一个故事开发代码**。这一方案提供了**实时解决问题**和**实时质量保证**的机制（在代码写出后及时得到复审），同时实施中**不同成员担任的角色略有不同**，也使得开发者能集中精力于手头的问题。

当结对的两人完成其工作，他们所开发的代码将**与其他人的工作集成起来**。**集成团队**的日常工作实施/**结对者自己负责集成**，这种“连续集成”策略有助于避免兼容性和接口问题，建立能及早发现错误的“冒烟测试”环境。

4. 测试

a. 编码开始之前的**单元测试**。所建立的单元测试应当使用一个**可以自动实施的框架**，支持代码修改之后即时的回归测试

b. 将个人的单元测试组织到一个“通用测试集”，每天都可以进行**系统的集成和确认测试**。目的是1为团队提供连续的进展指示，2一旦发生问题的时候及早提出预警，节省时间。

c. **XP验收测试/客户测试**。根据本次软件发布中所实现的用户故事而确定。由客户规定技术条件，并且着眼于客户可见的、可评审的系统级的特征和功能。

- **Scrum**：强调使用一系列已被证实行之有效的**软件过程模式**，允许团队根据项目需求灵活地构建自己的过程。
- 还有特征驱动开发 (FDD)、动态系统开发方法 (DSDM)、自适应软件开发 (ASD)等敏捷过程模型

敏捷宣言：

我们正在通过亲身实践以及帮助他人实践的方式来揭示更好的软件开发之路，通过这项工作，我们认识到：

个人和这些个人之间的交流胜过了开发过程和工具

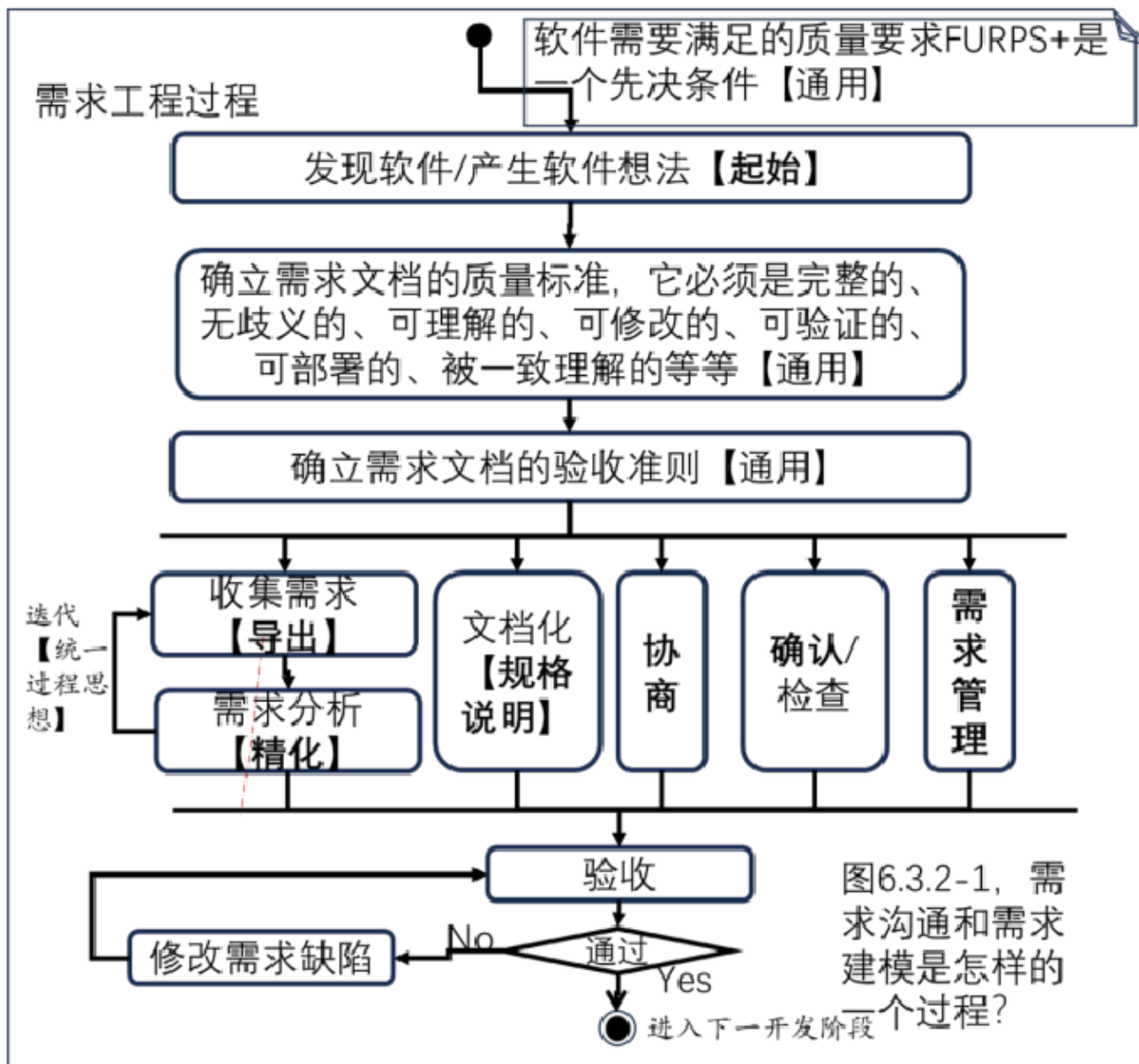
可运行的软件胜过了宽泛的文档

客户合作胜过了合同谈判

对变更的良好响应胜过了按部就班地遵循计划

也就是说，虽然上述右边的各项很有价值，但我们认为左边的各项具有更大的价值。

3. 关于需求工程的7项职责是什么？



1. 执行需求工程过程，将产生需求制品（称需求文档）。我们必须关注它们的质量，因为它是面向涉众的，且是后续开发活动的基准，一旦其质量出现问题，就可能偏离涉众的期望，从而软件失去价值。为此，需要在正式开始需求沟通和需求建模之间，先确立需求文档的质量标准和验收标准，以便行动具有明确的目标，使得创建行为——执行7项任务创建需求制品时，自然采用满足这些质量准则的正确流程和技术，而且也使得判断需求制品好坏——是否满足质量准则——提供判断标准，最终提供满足质量准则的需求制品。此处的质量标准具有普遍性，即任何软件项目皆如此。参见6.3.3理解需求文档的质量准则。

2. 需求工程的主线活动是：**发现软件（起始）、收集需求（导出）和需求分析（精化）**；辅线活动是**文档化（规格说明）、确认、协商和需求管理**。其得到有效实施的先决条件是确立需求文档的质量准则和验收准则。
3. 在整个时间线上，需求收集和需求分析是同时进行的；只是，在早期，焦点活动是“需求收集”，之后，焦点活动向“需求分析”转移。这正是统一过程思想的体现。
4. 无论发现软件，需求收集，还是需求分析，都要将活动的产物（称“需求制品”）用自然语言或模型语言将其陈述出来，这一活动称之为“规格说明”。
5. 无论发现软件，需求收集，还是需求分析，都要对“需求制品”（文档化的需求）的质量进行确认。不仅如此，还要对产生需求制品的过程质量进行确认，即检查你是否按照需求工程或软件工程的过程和方法进行的开发活动。
6. 一旦发生需求缺陷或冲突，则需要启动协商程序。
7. 除了规格说明（文档化）、确认活动是贯穿性活动，需求管理也是贯穿性活动。

4. 有人说,"直到程序开始运行,才能评估其质量。"你认为这说法正确吗?请说明理由。



Q1. 为什么不能直到程序开始运行，才评估其质量？

A1. 缺陷更多的发生在需求和设计上，如果直到程序开始运行，才评估其质量，那么，修复的代价无疑是巨大的。

Q2. 在程序开始运行前，做什么可以保证质量呢？

A2. 评审与测试是质量保证的两大手段。事实上，在软件开发的整个生命周期中，所有的制品以及创作这些制品的过程的质量，都需要评审。我们会通过需求评审、设计评审和代码走查，从项目早期开始持续发现缺陷，并修改缺陷；之后，随着编程活动的进行，我们会让软件运行于测试环境或真实的情形下，持续进行单元测试、集成测试、确认测试和系统测试。技术评审和测试都是非常有效的质量保证（评估）手段，其中技术评审被称之为“质量过滤器”，是比测试更加有效的质量保证（评估）手段。并且，经评审通过的软件规约，才能作为测试用例设计的依据。所以，评审是非常重要的、关键的质量保证手段。